

SECTION 12

APPLICATIONS INFORMATION

This section provides guidelines for using the MC68030. First, it discusses the requirements for adapting the MC68030 to MC68020 designs. Then, it describes the use of the MC68881 and MC68882 coprocessors with the MC68030. The byte select logic is described next, followed by memory interface information. A description of external caches, the use of the STATUS and REFILL signals, and power and ground considerations complete the section.

12.1 ADAPTING THE MC68030 TO MC68020 DESIGNS

Perhaps the easiest way to first utilize the MC68030 is in a system designed for the MC68020. This is possible due to the complete compatibility of the asynchronous buses of the MC68020 and MC68030. This section describes how to configure an adapter for the MC68030 to allow insertion into an existing MC68020-based system. Software and architectural differences between the two processors are also discussed. The need for an adapter is absolute because the MC68020 and MC68030 are NOT pin compatible. Use of the adapter board provides the immediate capability for evaluating the programmer's model and instruction set of the MC68030 and for developing software to utilize the MC68030's additional enhanced features. This adapter board also provides a relatively simple method for increasing the performance of an existing MC68020 or MC68020/MC68851 system by insertion of a more advanced 32-bit MPU with an on-chip data cache and an on-chip MMU. Since the adapter board does not support of the synchronous bus interface of the MC68030, performance measurements for the MC68030 used in this manner may be misleading when compared to a system designed specifically for the MC68030.

The adapter board plugs into the CPU socket of an MC68020 target system, drawing power, ground, and clock signals through the socket and running bus cycles in a fashion compatible with the MC68030. The only support hardware necessary is a single 1K-ohm pullup resistor and two capacitors for decoupling power and ground on the adapter board.

12.1.1 Signal Routing

Figure 12-1 shows the complete schematic for routing the signals of the MC68030 to the MC68020 header. All signals common to both processors are directly routed to the corresponding signal of the other processor. The signals on the MC68030 that do not have a compatible signal on the MC68020 are either pulled up or left unconnected:

Pulled Up

STERM
CBACK
CIIN
MMUDIS

No Connect

STATUS
REFILL
CBREQ
CIOUT

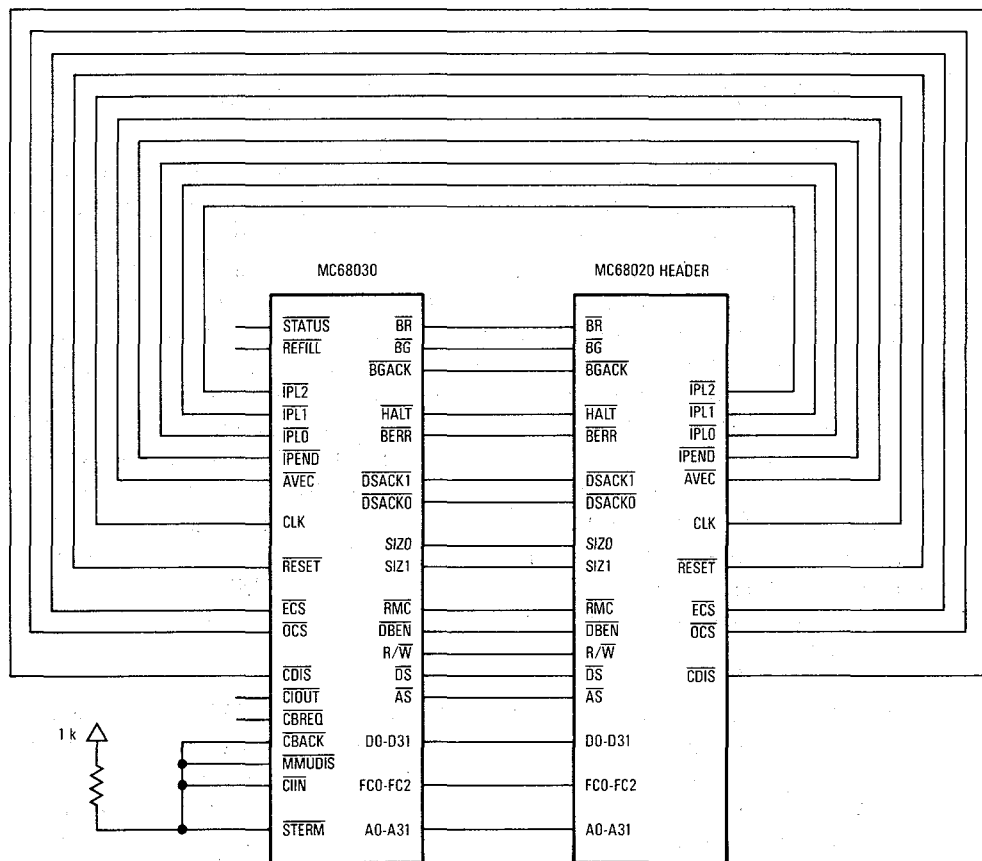


Figure 12-1. Signal Routing for Adapting the MC68030 to MC68020 Designs

12.1.2 Hardware Differences

Before enabling the on-chip caches of the MC68030, an important system feature must be checked. Because of the MC68030 cache organization and implementation, cachable read bus cycles are expected to transfer the entire port width of data (as indicated by the DSACKx encoding), regardless of how many bytes are actually requested by the SIz pins. The MC68020 did not have this requirement, and system memory banks or peripherals may or may not supply the amount of data required by the MC68030. If the target system does not supply the full port width with valid data for any cachable instruction or data access the user should either designate that area of memory as noncachable (with the MMU) or not enable the corresponding on-chip cache(s). In some systems, modifying the target system hardware may also be an option; frequently, the byte select logic is generated by a single PAL, which might easily be replaced or reprogrammed to select all bytes during read cycles from multibyte ports.

The HALT input-only signal of the MC68030 is slightly different than the bidirectional HALT signal of the MC68020. However, this should not cause any problems beyond eliminating an indication to the external system (e.g., lighting an LED) that the processor has halted due to a double bus fault.

When used in a system originally designed for both an MC68020 and an MC68851, the MC68851 may be left in the system or removed (and replaced with a jumpered header). However, if left in the system, the MC68851 is not accessible to the programmer with the M68000 coprocessor interface. All MMU instructions access the MC68030's on-chip MMU. This is true even if the MC68030's MMUDIS signal is asserted. The benefit in removing the MC68851 is that the minimum asynchronous bus cycle time to the physical bus is reduced from four clock cycles to three.

If the MC68851 is removed and replaced with a jumpered header, the following MC68851 signals may need special system-specific consideration: CLI, RMC, LBRO, LBG, LBGACK, and LBGI. During translation table searches, the MC68851 asserts the cache load inhibit (CLI) signal but not RMC; whereas,

the MC68030 asserts $\overline{RMC}$ but not $\overline{CIOUT}$. In simple MC68020/MC68851 systems without logical bus arbitration or logical caches, the MC68851's jumper can have the following signals connected together:

$\overline{LAS} \leftrightarrow \overline{PAS}$
 $\overline{LBRO} \leftrightarrow \overline{PBR}$
 $\overline{LBI} \leftrightarrow \overline{PBI}$
 $\overline{LBGACK} \leftrightarrow \overline{PBGACK}$
 $LA(8-31) \leftrightarrow PA(8-31)$
 $\overline{CLI} \leftrightarrow$ no connect or $\overline{LAS}$

$\overline{CLI}$ has two connection options because some systems may use $\overline{CLI}$ to qualify the occurrence of CPU space cycles since the MC68851's $\overline{PAS}$ does not assert.

12.1.3 Software Differences

The instruction cache control bits in the cache control register (CACR) of the MC68030 are in the identical bit positions as the corresponding bits as the MC68020's CACR. However, the MC68030 has additional control bits for burst enable and data cache control. Because this adapter board does not support synchronous bus cycles (and thus burst mode), enabling burst mode through the CACR does not affect system operation in any way. Refer to **SECTION 6 ON-CHIP CACHE MEMORIES** for more information on the bit positions and functions of the CACR bits.

When used in a system originally designed for an MC68020, a difference a programmer must be aware of is that the MC68030 does not support the CALLM and RTM instructions of the MC68020. If code is executed on the MC68030 using either the CALLM or RTM instructions, an unimplemented instruction exception is taken. If no MMU software development capability is desired and the cache behavior described under hardware differences is understood, the user may ignore the MC68030 MMU.

When the adapter is used in a system originally designed for the MC68020/MC68851 pair, the software differences described below also apply. The MC68030's MMU offers a subset of the MC68851 features. The features not supported by the MC68030 MMU are listed below:

- On-chip breakpoint registers
- Task aliasing
- Instructions: PBcc, PDBcc, PRESTORE, PSAVE, PSc, PTRAPcc, PVALID



Only control-alterable addressing modes are allowed for MMU instructions on the MC68030.

A feature new to the MC68030 MMU (and not on the MC68851) is the transparent translation of two logical address blocks with the transparent translation registers. See **SECTION 9 MEMORY MANAGEMENT UNIT**.

12.2 FLOATING-POINT UNITS

Floating-point support for the MC68030 is provided by the MC68881 floating-point coprocessor and the MC68882 enhanced floating-point coprocessor. Both devices offer a full implementation of the *IEEE Standard for Binary Floating-Point Arithmetic* (754). The MC68882 is a pin and software-compatible upgrade of the MC68881, with an optimized MPU interface that provides over 1.5 times the performance of the MC68881 at the same clock frequency.

Both coprocessors provide a logical extension to the integer data processing capabilities of the main processor. They contain a very high performance floating-point arithmetic unit and a set of floating-point data registers that are utilized in a manner that is analagous to the use of the integer data registers of the processor. The MC68881/MC68882 instruction set is a natural extension of all earlier members of the M68000 Family and supports all of the addressing modes and data types of the host MC68030. The programmer perceives the MC68030/coprocessor execution model as if both devices are implemented on one chip. In addition to supporting the full IEEE standard, the MC68881 and MC68882 provide a full set of trigonometric and transcendental functions, on-chip constants, and a full 80-bit extended-precision-real data format.

The interface of the MC68030 to the MC68881 or the MC68882 is easily tailored to system cost/performance needs. The MC68030 and the MC68881/MC68882 communicate via standard asynchronous M68000 bus cycles. All data transfers are performed by the main processor at the request of the MC68881/MC68882; thus memory management, bus errors, address errors, and bus arbitration function as if the MC68881/MC68882 instructions are executed by the main processor. The floating-point unit and the processor may operate at different clock speeds, and up to seven floating-point coprocessors may reside in an MC68030 system simultaneously.

Figure 12-2 illustrates the coprocessor interface connection of an MC68881/MC68882 to an MC68030 (uses entire 32-bit data bus). The MC68881/MC68882 is configured to operate with a 32-bit data bus when both the A0 and SIZE

pins are connected to V_{CC} . Refer to the MC68881UM/AD MC68881/MC68882 *Floating-Point Coprocessor User's Manual* for configuring the MC68881/MC68882 for smaller data bus widths. Note that the MC68030 cache inhibit input ($\overline{CIIN}$) signal is not used for the coprocessor interface because the MC68030 does not cache data obtained during CPU space accesses.

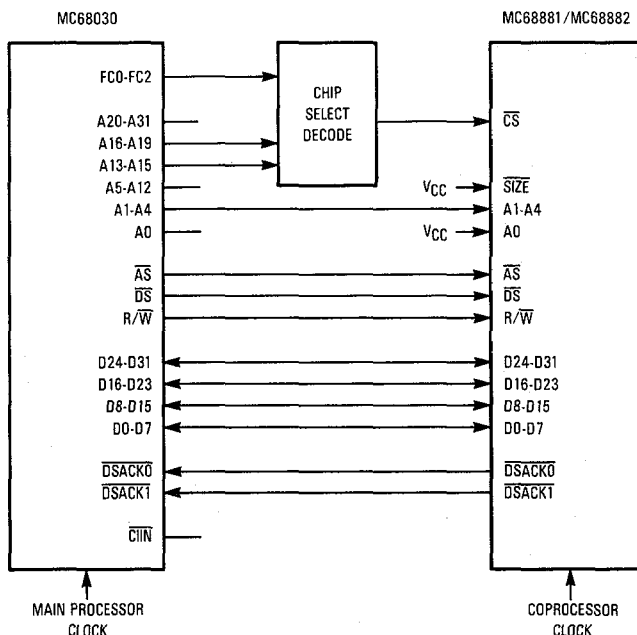


Figure 12-2. 32-Bit Data Bus Coprocessor Connection

The chip select ($\overline{CS}$) decode circuitry is asynchronous logic that detects when a particular floating-point coprocessor is addressed. The MC68030 signals used by the logic include the function code signals (FC0–FC2), and the address lines (A13–A19). Refer to **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION** for more information concerning the encoding of these signals. All or just a subset of these lines may be decoded depending on the number of coprocessors in the system and the degree of redundant mapping allowed in the system.

The major concern of a system designer is to design a $\overline{CS}$ interface that meets the AC electrical specifications for both the MC68030 (MPU) and the MC68881/MC68882 (FPCP) without adding unnecessary wait states to FPCP accesses. The following maximum specifications (relative to CLK low) meet these objectives:

$$t_{CLK \text{ low to } \overline{AS} \text{ low}} \leq (\text{MPU Spec 1} - \text{MPU Spec 47A} - \text{FPCP Spec 19}) \quad (1)$$

$$t_{CLK \text{ low to } \overline{CS} \text{ low}} \leq (\text{MPU Spec 1} - \text{MPU Spec 47A} - \text{FPCP Spec 19}) \quad (2)$$

Even though requirement (1) is not met under worst case conditions, if the MPU $\overline{AS}$ is loaded within specifications and the $\overline{AS}$ input to the FPCP is unbuffered, the requirement is met under typical conditions. Designing the $\overline{CS}$ generation circuit to meet requirement (2) provides the highest probability that accesses to the FPCP occur without unnecessary wait states. A PAL 16L8 (see Figure 12-3) with a maximum propagation delay of 10 ns, programmed according to the equations in Figure 12-4, can be used to generate $\overline{CS}$. For a 25-MHz system, $t_{CLK \text{ low to } \overline{CS} \text{ low}}$ is less than or equal to 10 ns when this design is used. Should worst case conditions cause $t_{CLK \text{ low to } \overline{AS} \text{ low}}$ to exceed requirement (1), one wait state is inserted in the access to the FPCP; no other adverse effect occurs. Figure 12-5 shows the bus cycle timing for this interface. Refer to MC68881UM/AD, *MC68881/MC68882 Floating-Point Coprocessor User's Manual*, for FPCP specifications.

The circuit that generates $\overline{CS}$ must meet another requirement. When a non-floating-point access immediately follows a floating-point access, $\overline{CS}$ (for the floating-point access) must be negated before $\overline{AS}$ and $\overline{DS}$ (for the subsequent access) are asserted. The PAL circuit previously described also meets this requirement.

For example, if a system has only one coprocessor, the full decoding of the ten signals (FC0–FC2 and A13–A19) provided by the PAL equations in Figure 12-4 is not absolutely necessary. It may be sufficient to use only FC0–FC1 and A16–A17. FC0–FC1 indicate when a bus cycle is operating in either CPU space (\$7) or user-defined space (\$3), and A16–A17 encode CPU space type as coprocessor space (\$2). A13–A15 can be ignored in this case because they encode the coprocessor identification code (CpID) used to differentiate between multiple coprocessors in a system. Motorola assemblers always default to a CpID of \$1 for floating-point instructions; this can be controlled with assembler directives if a different CpID is desired or if multiple coprocessors exist in the system.

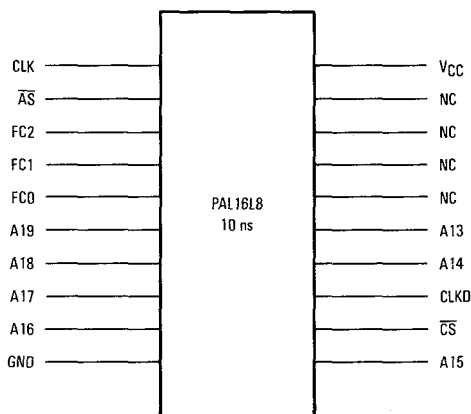


Figure 12-3. Chip-Select Generation PAL

PAL16I8

FPCP CS GENERATION CIRCUITRY FOR 25 MHz OPERATION

MOTOROLA INC., AUSTIN, TEXAS

CLK	AS	FC2	FC1	FC0	A19	A18	A17	A16	GND
A15	/CS	/CLKD	A14	A13	NC	NC	NC	NC	VCC

CS	= FC2	* FC1	* FC0						
	* /A19	* /A18	* A17	* /A16					;cpu space = \$7
	* /A15	* /A14	* A13						;coprocessor access = \$2
	* /CLK								;coprocessor id = \$1
									;qualified by MPU clock low
+	FC2	* FC1	* FC0						;cpu space = \$7
	* /A19	* /A18	* A17	* /A16					;coprocessor access = \$2
	* /A15	* /A14	* A13						;coprocessor id = \$1
	* /AS								;qualified by address strobe low
+	FC2	* FC1	* FC0						
	* /A19	* /A18	* A17	* /A16					;coprocessor access = \$2
	* /A15	* /A14	* A13						;coprocessor id = \$1
	* /CLKD								;qualified by CLKD (delayed CLK)

CLKD = CLK

Description: There are three terms to the CS generation. The first term denotes the earliest time CS can be asserted. The second term is used to assert CS until the end of the FPCP access. The third term is to ensure that no race condition occurs in case of a late AS.

Figure 12-4. PAL Equations

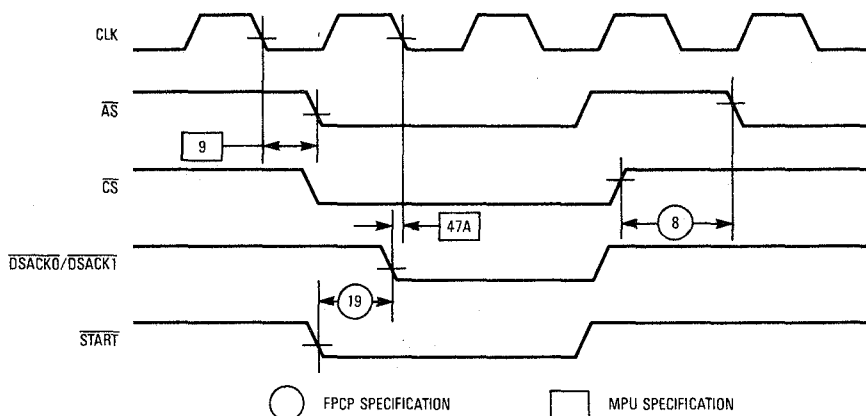


Figure 12-5. Bus Cycle Timing Diagram

12.3 BYTE SELECT LOGIC FOR THE MC68030

The architecture of the MC68030 allows it to support byte, word, and long-word operand transfers to any 8-, 16-, or 32-bit data port regardless of alignment. This feature allows the programmer to write code that is not bus-width specific. When accessed, the peripheral or memory subsystem reports its actual port size to the processor, and the MC68030 then dynamically sizes the data transfer accordingly, using multiple bus cycles when necessary. Hardware designers also have the flexibility to choose implementations independent of software prejudices. The following paragraphs describe the generation of byte select control signals that enable the dynamic bus sizing mechanism, the transfer of differently sized operands, and the transfer of misaligned operands to operate correctly.

The following signals control the MC68030 operand transfer mechanism:

- A1, A0 = Address lines. The most significant byte of the operand to be transferred is addressed directly.
- SI21, SI20 = Transfer size. Output of the MC68030. These indicate the number of bytes of an operand remaining to be transferred during a given bus cycle.
- $\overline{R/W}$ = Read/Write. Output of the MC68030. For byte select generation in MC68030 systems, $\overline{R/W}$ must be included in the logic if the data from the device is cachable.

- $\overline{\text{DSACK1}}, \overline{\text{DSACK0}}$ = Data transfer and size acknowledge. Driven by an asynchronous port to indicate the actual bus width of the port.
- $\overline{\text{STERM}}$ = Synchronous termination. Driven by a 32-bit synchronous port only.

The MC68030 assumes that 16-bit ports are situated on data lines D16–D31, and that 8-bit ports are situated on data lines D24–D31. This ensures that the following logic works correctly with the MC68030's on-chip internal-to-external data bus multiplexer. Refer to **SECTION 7 BUS OPERATION** for more details on the dynamic bus sizing mechanism.

The need for byte select signals is best illustrated by an example. Consider a long-word write cycle to an odd address in word-organized memory. The transfer requires three bus cycles to complete. The first bus cycle transfers the most significant byte of the long word on D16–D23. The second bus cycle transfers a word on D16–D31, and the last bus cycle transfers the least significant byte of the original long word on D24–D31. In order not to overwrite those bytes which are not used these transfers, a unique byte data strobe must be generated for each byte when using devices with 16- and 32-bit port widths.

For noncachable read cycles and all write cycles, the required active bytes of the data bus for any given bus transfer are a function of the size (SIZ0/SIZ1) and lower address (A0/A1) outputs and are shown in Table 12-1. Individual strobes or select signals can be generated by decoding these four signals for every bus cycle. Devices residing on 8-bit ports can utilize data strobe ($\overline{\text{DS}}$) alone since there is only one valid byte for any transfer.

During cachable read cycles, the addressed device must provide valid data over its full bus width (as indicated by $\overline{\text{DSACKx}}$ or $\overline{\text{STERM}}$). While instructions are always prefetched as long-word-aligned accesses, data fetches can occur with any alignment and size. Because the MC68030 assumes that the entire data bus port size contains valid data, cachable data read bus cycles must provide as much data as signaled by the port size during a bus cycle. To satisfy this requirement, the R/W signal must be included in the byte select logic for the MC68030.

Figure 12-6 shows a block diagram of an MC68030 system with two memory banks. The PAL provides memory-mapped byte select signals for an asynchronous 32-bit port and unmapped byte select signals for other memory banks or ports. Figure 12-7 provides sample equations for the PAL.

Table 12-1. Data Bus Activity for Byte, Word, and Long-Word Ports

Transfer Size	SIZ1	SIZ0	A1	A0	Data Bus Active Sections Byte (B) – Word (W) – Long-Word (L) Ports			
					D31-D24	D23-D16	D15-D8	D7-D0
Byte	0	1	0	0	B W L	—	—	—
	0	1	0	1	B	W L	—	—
	0	1	1	0	B W	—	L	—
	0	1	1	1	B	W	—	L
Word	1	0	0	0	B W L	W L	—	—
	1	0	0	1	B	W L	L	—
	1	0	1	0	B W	W	L	L
	1	0	1	1	B	W	—	L
Three Byte	1	1	0	0	B W L	W L	L	—
	1	1	0	1	B	W L	L	L
	1	1	1	0	B W	W	L	L
	1	1	1	1	B	W	—	L
Long Word	0	0	0	0	B W L	W L	L	L
	0	0	0	1	B	W L	L	L
	0	0	1	0	B W	W	L	L
	0	0	1	1	B	W	—	L

The PAL equations and circuits presented here are not intended to be the optimal implementation for every system. Depending on the CPU's clock frequency, memory access times, and system architecture, different circuits may be required.

12.4 MEMORY INTERFACE

The MC68030 is capable of running three types of external bus cycles as determined by the cycle termination and handshake signals (refer to **SECTION 7 BUS OPERATION**). These three types of bus cycles are:

1. Asynchronous cycles, terminated by the $\overline{\text{DSACKx}}$ signals, have a minimum duration of three processor clock periods in which up to four bytes are transferred.
2. Synchronous cycles, terminated by the $\overline{\text{STERM}}$ signal, have a minimum duration of two processor clock periods in which up to four bytes are transferred.
3. Burst operation cycles, terminated by the $\overline{\text{STERM}}$ and $\overline{\text{CBACK}}$ signals, have a duration of as little as five processor clock periods in which up to four long words (16 bytes) are transferred.

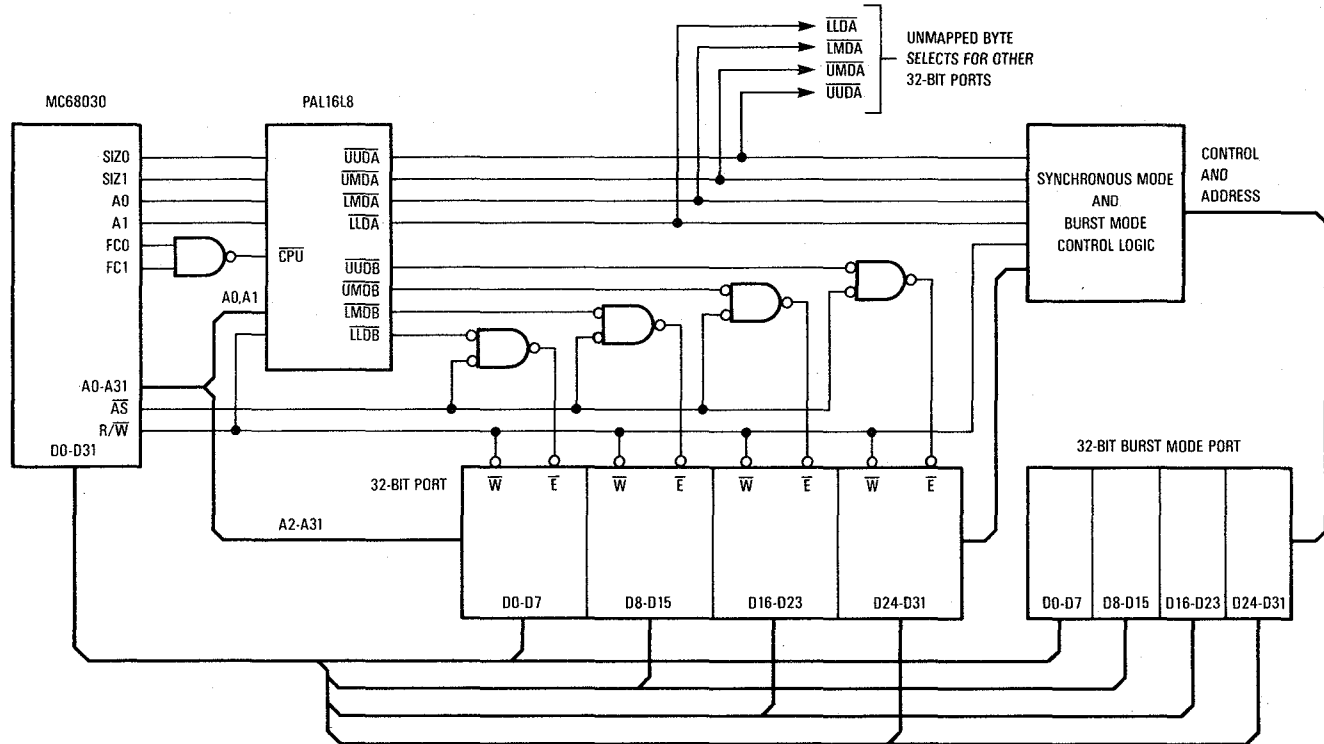


Figure 12-6. Example MC68030 Byte Select PAL System Configuration

PAL16L8

U1

MC68030 BYTE DATA SELECT GENERATION FOR 32-BIT PORTS, MAPPED AND UNMAPPED.

MOTOROLA INC., AUSTIN, TEXAS

A0	A1	SIZ0	SIZ1	RW	A18	A19	A20	A21	GND
/CPU	/UUDA	/UMDA	/LMDA	/LLDA	/UUDA	/UMDB	/LMDB	/LLDB	VCC

UUDA = RW	;enable upper byte on read of 32-bit port
+ /A0 * /A1	;directly addressed, any size
UMDA = RW	;enable upper middle byte on read of 32-bit port
+ A0 * /A1	;directly addressed, any size
+ /A1 * /SIZ0	;word aligned, size byte or three byte
+ /A1 * SIZ1	;word aligned, size is word or long word
LMDA = RW	;enable lower middle byte on read of 32-bit port
+ /A0 * A1	;directly addressed, any size
+ /A1 * /SIZ0 * /SIZ1	;word aligned, size is long word
+ /A1 * SIZ0 * SIZ1	;word aligned, size is three byte
+ /A1 * A0 * /SIZ0	;word aligned, size is word or long word
LLDA = RW	;enable lower byte on read of 32-bit port
+ A0 * A1	;directly addressed, any size
+ A0 * SIZ0 * SIZ1	;odd alignment, three byte size
+ /SIZ0 * /SIZ1	;size is long word, any address
+ A1 * SIZ1	;word aligned, word or three byte size
UUSB = RW * /CPU * (addressb)	;enable upper byte on read of 32-bit port
+ /A0 * /A1 * /CPU * (addressb)	;directly addressed, any size
UMDB = RW * /CPU * (addressb)	;enable upper middle byte on read of 32-bit port
+ A0 * /A1 * /CPU * (addressb)	;directly addressed, any size
+ /A1 * /SIZ0 * /CPU * (addressb)	;word aligned, size byte or three byte
+ /A1 * SIZ1 * /CPU * (addressb)	;word aligned, size is word or long word
LMDB = RW * /CPU * (addressb)	;enable lower middle byte on read of 32-bit port
+ /A0 * A1 * /CPU * (addressb)	;directly addressed, any size
+ /A1 * /SIZ0 * /SIZ1 * /CPU * (addressb)	;word aligned, size is long word
+ /A1 * SIZ0 * SIZ1 * /CPU * (addressb)	;word aligned, size is three byte
+ /A1 * A0 * /SIZ0 * /CPU * (addressb)	;word aligned, size is word or long word
LLDB = RW * /CPU * (addressb)	;enable lower byte on read of 32-bit port
+ A0 * A1 * /CPU * (addressb)	;directly addressed, any size
+ A0 * SIZ0 * SIZ1 * /CPU * (addressb)	;odd alignment, three byte size
+ /SIZ0 * /SIZ1 * /CPU * (addressb)	;size is long word, any address
+ A1 * SIZ1 * /CPU * (addressb)	;word aligned, word or three byte size

DESCRIPTION: Byte select signals for writing. On reads, all bytes selects are asserted if the respective memory block is addressed. The input signal /CPU prevents byte select assertion during CPU space cycles and is derived from NANDing FC0-FC1 or FC0-FC2. The label, (addressb), is a designer-selectable combination of address lines used to generate the proper address decode for the system's memory bank. With the address lines given here the decode block size is 256K bytes. A similar address might be included in the equations for UUDA, UMDA, etc. if the designer wishes them to be memory mapped also.

Figure 12-7. MC68030 Byte Select PAL Equations

During read operations, M68000 processors latch data on the last falling clock edge of the bus cycle, one-half clock before the bus cycle ends (burst mode is a special case). Latching data here, instead of the next rising clock edge, helps to avoid data bus contention with the next bus cycle and allows the MC68030 to receive the data into its execution unit sooner for a net performance increase.

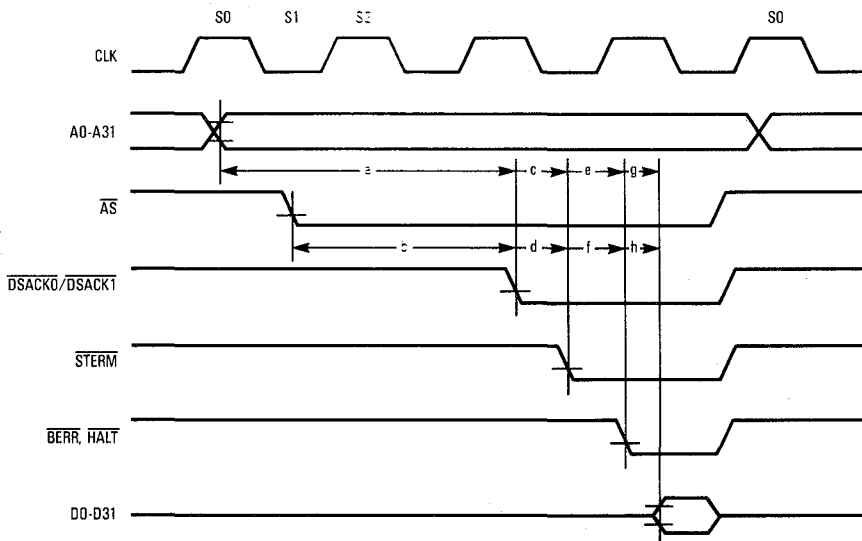
Write operations also use this data bus timing to allow data hold times from the negating strobes and to avoid any bus contention with the following bus cycle. This usually allows the system to be designed with a minimum of bus buffers and latches.

One of the benefits of the MC68030's on-chip caches is that the effect of external wait states on performance is lessened because the caches are always accessed in fewer than "no wait states", regardless of the external memory configuration. This feature makes the MC68030 (and MC68020) unique among other general-purpose microprocessors.

12.4.1 Access Time Calculations

The timing paths that are typically critical in any memory interface are illustrated and defined in Figure 12-8. For burst transfers, the first long word transferred also uses these parameters, but the subsequent transfers are different and are discussed in **12.4.2 Burst Mode Cycles**.

The type of device that is interfaced to the MC68030 determines exactly which of the paths is most critical. The address-to-data paths are typically the critical paths for static devices since there is no penalty for initiating a cycle to these devices and later validating that access with the appropriate bus control signal. Conversely, the address-strobe-to-data-valid path is often most critical for dynamic devices since the cycle must be validated before an access can be initiated. For devices that signal termination of a bus cycle before data is validated (e.g., error detection and correction hardware and some external caches) to improve performance, the critical path may be from the address or strobes to the assertion of $\overline{BERR}$ (or $\overline{BERR}$ and $\overline{HALT}$). Finally, the address-valid-to- $\overline{DSACKx}$ -or- $\overline{STERM}$ -asserted path is most critical for very fast devices and external caches, since the time available between when the address is valid and when $\overline{DSACKx}$ or $\overline{STERM}$ must be asserted to terminate the bus cycle is minimal. Table 12-2 provides the equations required to calculate the various memory access times assuming a 50-percent duty cycle clock.



NOTE: This diagram illustrates access time calculations only. $\overline{\text{DSACK0}}/\overline{\text{DSACK1}}$ and $\overline{\text{STERM}}$ should never be asserted together during the same bus cycle.

Parameter	Description	System	Equation
a	Address Valid to $\overline{\text{DSACKx}}$ Asserted	tAVDL	12-1
b	Address Strobe Asserted to $\overline{\text{DSACKx}}$ Asserted	tSADL	12-2
c	Address Valid to $\overline{\text{STERM}}$ Asserted	tAVSL	12-3
d	Address Strobe Asserted to $\overline{\text{STERM}}$ Asserted	tSASL	12-4
e	Address Valid to $\overline{\text{BERR/HALT}}$ Asserted	tAVBHL	12-5
f	Address Strobe Asserted to $\overline{\text{BERR/HALT}}$ Asserted	tSABHL	12-6
g	Address Valid to Data Valid	tAVDV	12-7
h	Address Strobe Asserted to Data Valid	tSADV	12-8

Figure 12-8. Access Time Computation Diagram

During asynchronous bus cycles, $\overline{\text{DSACK1}}$ and $\overline{\text{DSACK0}}$ are used to terminate the current bus cycle. In true asynchronous operations, such as accesses to peripherals operating at a different clock frequency, either or both signals may be asserted without regard to the clock, and then data must be valid a certain amount of time later as defined by specification #31. With a 16.67-MHz processor, this time is 50 ns after $\overline{\text{DSACKx}}$ asserts; with a 20.0-MHz processor, this time is 43 ns after $\overline{\text{DSACK}}$ asserts (both numbers vary with the actual clock frequency).

Table 12-2. Memory Access Time Equations at 20 MHz

	N=2	N=3	N=4	N=5	N=6
(12-1) $t_{AVDL} = (N-1) \cdot t_1 - t_2 - t_6 - t_{47A}$	—	46 ns	96 ns	146 ns	196 ns
(12-2) $t_{SADL} = (N-2) \cdot t_1 - t_9 - t_{47A}$	—	26 ns	76 ns	126 ns	176 ns
(12-3) $t_{AVSL} = (N-1) \cdot t_1 - t_6 - t_{60}$	21 ns	71 ns	121 ns	171 ns	221 ns
(12-4) $t_{SASL} = (N-1) \cdot t_1 - t_3 - t_9 - t_{60}$	1 ns	51 ns	101 ns	151 ns	201 ns
(12-5) $t_{AVBHL} = N \cdot t_1 - t_2 - t_6 - t_{27A}$	40 ns	90 ns	140 ns	190 ns	240 ns
(12-6) $t_{SABHL} = (N-1) \cdot t_1 - t_9 - t_{27A}$	20 ns	70 ns	120 ns	170 ns	220 ns
(12-7) $t_{ADV} = N \cdot t_1 - t_2 - t_6 - t_{27}$	46 ns	96 ns	146 ns	196 ns	246 ns
(12-8) $t_{ADV} = (N-1) \cdot t_1 - t_9 - t_{27}$	26 ns	76 ns	126 ns	176 ns	226 ns

where:

- tX = Refers to AC Electrical Specification #X
- t1 = The Clock Period
- t2 = The Clock Low Time
- t3 = The Clock High Time
- t6 = The Clock High to Address Valid Time
- t9 = The Clock Low to $\overline{AS}$ Low Delay
- t27 = The Data-In to Clock Low Setup Time
- t27A = The BERR/HALT to Clock Low Setup Time
- t47A = The Asynchronous Input Setup Time
- t60 = The Synchronous Input to CLK High Setup Time
- N = The Total Number of Clock Periods in the Bus Cycle (Nonburst)
($N \geq 2$ for Synchronous Cycles; $N \geq 3$ for Asynchronous Cycles)

However, many local memory systems do not operate in a truly asynchronous manner because the memory control logic can either be related to the MC68030's clock or worst case propagation delays are known; thus, asynchronous setup times for the DSACKx signals can be guaranteed. The timing requirements for this pseudo-synchronous DSACKx generation is governed by the equation for t_{AVDL} .

Synchronous cycles use the $\overline{STERM}$ signal to terminate the current bus cycle. In bus cycles of equal length, $\overline{STERM}$ has more relaxed timing requirements than DSACKx since an additional 30 ns is available when comparing t_{AVSL} (or t_{SASL}) to t_{AVDL} (or t_{SADL}). The only additional restriction is that $\overline{STERM}$ must meet the setup and hold times as defined by specifications #60 and #61, respectively, for all rising edges of the clock during a bus cycle. The value for t_{SASL} when the total number of clock periods (N) equals two in Table 12-2 requires further explanation. Because the calculated value of this access time (see Equation 12-4 of Table 12-2) is zero under certain conditions, hardware cannot always qualify $\overline{STERM}$ with $\overline{AS}$ at all frequencies. However, such qualification is not a requirement for the MC68030. $\overline{STERM}$ can be generated by the assertion of $\overline{ECS}$, the falling edge of S0, or most simply by the output(s) of an address decode or comparator logic. Note that other

devices in the system may require qualification of the access with $\overline{AS}$ since the MC68030 has the capability to initiate bus cycles and then abort them before the assertion of $\overline{AS}$.

Another way to optimize the CPU to memory access times in a system is to use a clock frequency less than the rated maximum of the specific MC68030 device. Table 12-3 provides calculated t_{AVDV} (see Equation 12-7 of Table 12-2) results for an MC68030RC16 and MC68030RC20 operating at various clock frequencies. If the system uses other clock frequencies, the above equations can be used to calculate the exact access times.

Table 12-3. Calculated t_{AVDV} Values for Operation at Frequencies Less Than or Equal to the CPU Maximum Frequency Rating

Equation 12-7 t_{AVDV}		MC68030RC20		MC68030RC25		
Clocks Per Bus Cycle (N) and Type	Wait States	Clock at 16.67 MHz	Clock at 20 MHz	Clock at 16.67 MHz	Clock at 20 MHz	Clock at 25 MHz
2 Clock Synchronous	0	61	46	68	53	38
3 Clock Synchronous	1	121	96	128	103	78
3 Clock Asynchronous	0	121	96	128	103	78
4 Clock Synchronous	2	181	146	188	153	118
4 Clock Asynchronous	1	181	146	188	153	118
5 Clock Synchronous	3	241	196	248	203	158
5 Clock Asynchronous	2	241	196	248	203	158
6 Clock Synchronous	4	301	246	308	253	198
6 Clock Asynchronous	3	301	246	308	253	198

12.4.2 Burst Mode Cycles

The memory access times for burst mode bus cycles follow the above equations for the first access only. For the subsequent (second, third, and fourth) accesses, the memory access time calculations depend on the architecture of the burst mode memory system.

Architectural tradeoffs include the width of the burst memory and the type of memory used. If the memory is 128 bits wide, the subsequent operand accesses do not affect the critical timing paths. For example, if a 3-1-1-1 burst accesses 128-bit-wide memory, the first access is governed by the equations in Table 12-2 for N equal to three. The subsequent accesses also use these values as a base but have additional clock periods added in. The second

access has one additional clock period, the third access has two additional clock periods, and the fourth has three additional clock periods. Thus, the access time for the first cycle determines the critical timing paths.

Memory that is 64 bits wide presents a compromise between the two configurations listed above.

12.5 STATIC RAM MEMORY BANKS

When the MC68030 is operating at a high clock frequency, a no-wait-state external memory system will most likely be composed of static RAMs. The following paragraphs discuss three static memory banks, which may be used as shown or as a starting point for an external cache design. The designs offer different levels of performance, bus utilization, and cost.

12.5.1 A Two-Clock Synchronous Memory Bank Using SRAMs

The MC68030 normally attains its highest performance when the external memory system can support a two-clock synchronous bus protocol. This section describes a complete memory bank containing 64K bytes that can operate with a 20-MHz MC68030 using two-clock accesses. Also discussed are several options and minor alterations to reduce cost or power consumption.

Figure 12-9 shows the complete memory bank and its connection to the MC68030. As drawn, the required parts include:

- (8) 16K $\times$ 4 SRAMs, 35-ns access time with separate I/O pins
- (4) 74F244 buffers
- (2) 74F32 OR gates
- (1) PAL16L8D (or equivalent)

The system must also provide any $\overline{\text{STERM}}$ consolidation circuitry as required (e.g., by the presence of multiple synchronous memory banks or ports). In Figure 12-9, this consolidation circuitry is shown as an AND gate.

The memory bank can be divided into three sections:

1. The byte select and address decode section (provided by the PAL),
2. The actual memory section (SRAMs), and
3. The buffer section.

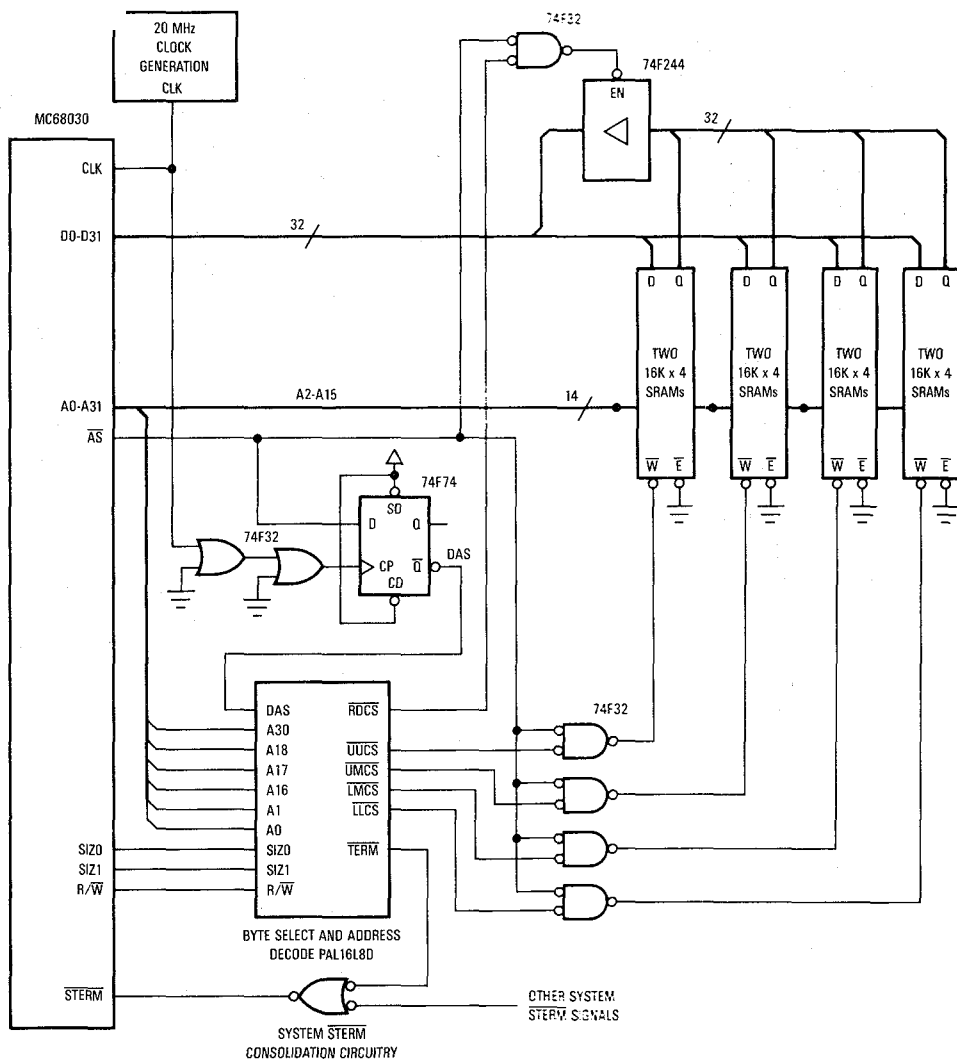


Figure 12-9. Example Two-Clock Read, Three-Clock Write Memory Bank

The first section consists of two 74F32 OR gates, a 74F74 D-type flip-flop, and a PAL16L8D. Example PAL equations are provided in Figure 12-10. The PAL generates six memory-mapped signals; four byte select signals for write operations, a buffer control signal, and the cycle termination signal. The byte select signals are only asserted during write operations when the processor is addressing the 64K bytes contained in the memory bank, and then only

when the appropriate byte (or bytes) is being written to as indicated by the SIZ_0 , SIZ_1 , A_0 , and A_1 signals. The four signals, $\overline{UUCS}$, $\overline{UMCS}$, $\overline{LMCS}$, and $\overline{LLCS}$, control data bits D_{24-D31} , D_{16-D23} , D_8-D_{15} , and D_0-D_7 , respectively. $\overline{AS}$ is used to qualify the byte select signals to avoid spurious writes to memory before the address is valid. During read operations, the read chip select ($\overline{RDCS}$) signal, qualified with $\overline{AS}$, controls the data buffers only (since the memory is already enabled with its $\overline{E}$ input grounded). The last signal generated by the PAL is the $\overline{TERM}$ signal. As its equation shows, $\overline{TERM}$ consists of two events: one for read cycles and the other for write cycles. For read cycles, $\overline{TERM}$ is an address decode signal that is asserted whenever the address corresponds to the encoded memory-mapped bank of SRAM. For write operations, a delayed form of $\overline{AS}$ ($\overline{DAS}$) is used to qualify the same address decode, which lengthens write operations to three clock cycles. The $\overline{DAS}$ signal generation is delayed from the clock edge by running the clock signal through two 74F32 OR gates before connecting to the 74F74 D-type flip-flop. This guarantees that the maximum propagation delay to generate the $\overline{TERM}$ signal does not violate the synchronous input hold time of the MC68030. By increasing write operation to three clock cycles, the MC68030 can easily meet the specified data setup time to the SRAMs before the negation of the write strobes ($\overline{W}$). $\overline{TERM}$ is then connected to the system's $\overline{STERM}$ consolidation circuitry. The consolidation circuitry should have no more than 15 ns of propagation delay. If the system has no other synchronous memory or ports, $\overline{TERM}$ may be connected directly to $\overline{STERM}$.

$UUCS = /A_0 * /A_1 \ 8 /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$	;directly addressed, any size
$UMCS = A_0 * /A_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ /A_1 * /SIZ_0 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ /A_1 * SIZ_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$	;directly addressed, any size ;word aligned, size byte or three byte ;word aligned, size is word or long word
$LMCS = /A_0 * A_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ /A_1 * /SIZ_0 * /SIZ_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ /A_1 * SIZ_0 * SIZ_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ /A_1 * A_0 * /SIZ_0 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$	;directly addressed, any size ;word aligned, size is long word ;word aligned, size is three byte ;word aligned, size is word or long word
$LLCS = A_0 * A_1 /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ A_0 * SIZ_0 * SIZ_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ /SIZ_0 * /SIZ_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$ $+ A_1 * SIZ_1 * /RW * /A_{16}*/A_{17}*/A_{18}*A_{30}*$	;directly addressed, any size ;odd alignment, three byte size ;size is long word, any address ;word aligned, word or three byte size
$RDCS = /A_{16}*/A_{17}*/A_{18}*A_{30}*RW$ $+ /A_{15} * /A_{17} * /A_{18}*A_{30} * /RW * DAS$	;immediate $\overline{STERM}$ with proper address ;write cycles take three clocks

DESCRIPTION: Byte select signals. The byte select signals are asserted only during write operations when the particular byte is being written. The synchronous bank of memory is always enabled, and writes are controlled by $\overline{W}$ on the memory. $\overline{RDCS}$ is for buffer control and only asserts for read operations. $\overline{TERM}$ is the cycle termination signals to the MC68030.

Figure 12-10. Example PAL Equations for Two-Clock Memory Bank

The second section contains the memory devices. Eight devices are used, but some designs may wish to increase this to support EDAC or to increase density. The most important feature of the memory devices used in this design is the separate data-in and data-out pins, which allow the SRAMs to be enabled before address decode is complete without causing data bus contention. The enable pins on the SRAMs have been grounded for both simplicity and improved memory access timing. If the designer wishes to include some type of enable circuitry to take advantage of low bus utilization for lower power consumption, the timing in this design will be preserved if the memory's $\overline{E}$ signal is asserted before the falling edge of state S0 (at the same time as or before the address becomes valid). Two possible enable circuits are shown in Figure 12-11.

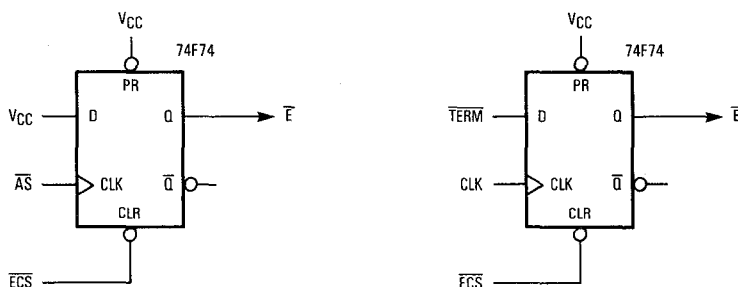


Figure 12-11. Additional Memory Enable Circuits

The third section of the memory bank is the data buffers. The data buffers are shown as 74F244, but 74AS244s may also be used. The $\overline{RD}\overline{CS}$ signal, qualified with $\overline{AS}$, controls the data buffers during read operations as described above.

To maximize performance, both read and write operations should be capable of completing in two clock cycles. Figure 12-12 shows a two-clock read and write memory bank. The required parts include:

- (8) 16K $\times$ 4 SRAMs, 25-ns access time with separate I/O pins
- (4) 74F244 buffers
- (2) 74F32 OR gates
- (1) PAL16L8D (or equivalent)
- (1) 74F74 D-type flip-flop
- (2) 74F373 transparent latches
- (1) 74AS21 AND gate
- (1) 74F04 inverter

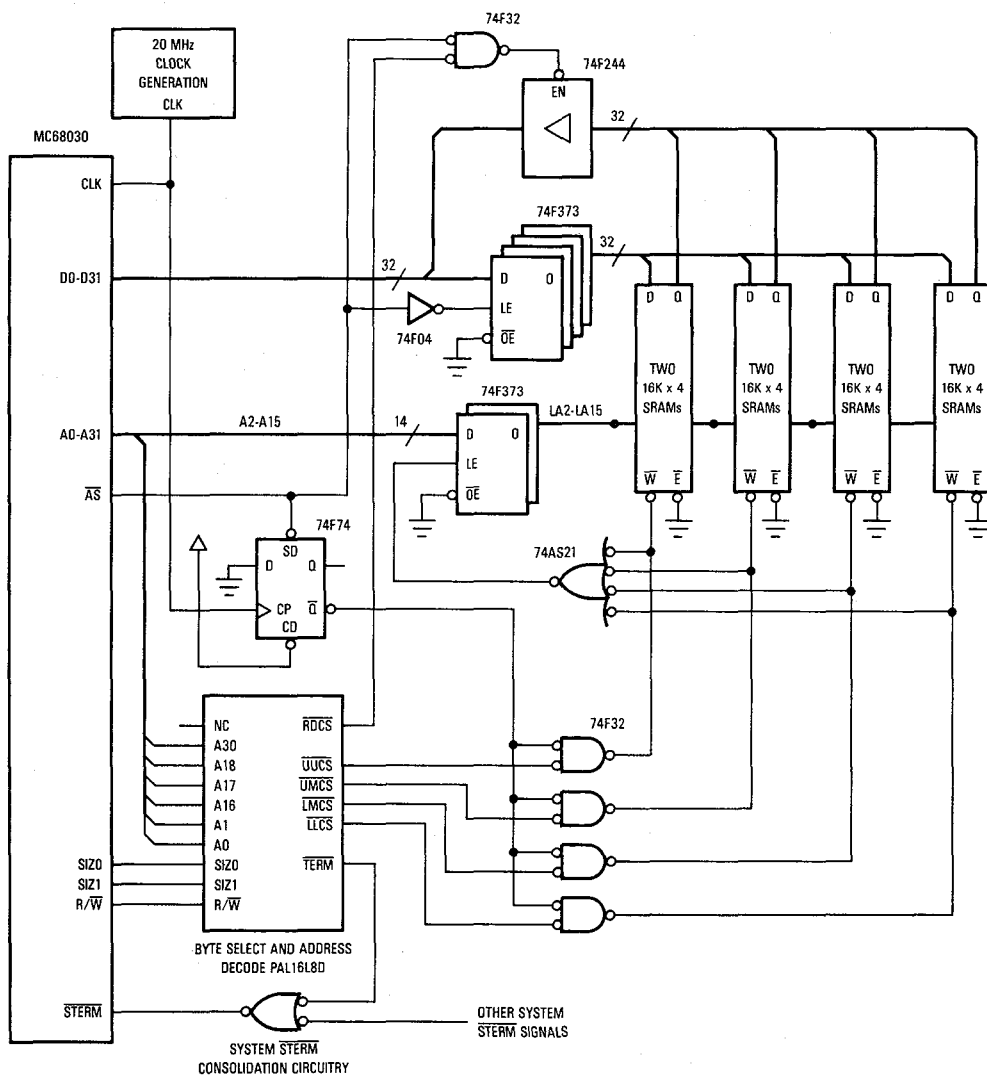


Figure 12-12. Example Two-Clock Read and Write Memory Bank

The structure of this design is very similar to the previous design and can similarly be divided into three main sections:

1. The byte select and address decode section (provided by the PAL).
2. The actual memory section (SRAMs).
3. The buffer/latch section (address and data).

The same PAL equations listed in Figure 12-10 are used with the exception of the $\overline{\text{TERM}}$ signal. Figure 12-13 shows the equation for $\overline{\text{TERM}}$, which is used by the two clock read and write design.

$\text{TERM} = /A16 * /A17 * /A16 * A30 ; \text{immediate } \overline{\text{STERM}}$ for both reads and writes

Figure 12-13. Example PAL Equation for Two-Clock Read and Write Memory Bank

$\overline{\text{TERM}}$ is simply an address decode signal in this design because both read and write operations complete in two clock periods. The other signals generated by the PAL have already been discussed in the previous design and are not repeated here. A latched version of $\overline{\text{AS}}$ is generated by a 74F74 D-type flip-flop and used to qualify the individual byte select signals from the PAL. The required SRAM data setup time on write cycles is ensured by keeping the write strobes ($\overline{\text{W}}$) active to the SRAMs until the rising edge of the clock that completes the MC68030 write operation.

The memory section in this design uses 25-ns SRAMs rather than the 35-ns SRAMs used in the previous design. The faster SRAMs compensate for the 74F373 transparent latches used on the address lines. Since the memory write operations complete after the MC68030 write bus cycle, both address and data are latched and held valid to the SRAMs until the write strobes ($\overline{\text{W}}$) negate. During read operations, the transparent latches on the address lines remain in the transparent mode, allowing the SRAMs to provide data through the 74F244 buffers in time to meet the specified data setup time to the MC68030.

Not all systems require the performance of 20-MHz two-clock bus cycles, nor will all systems be able to afford the fast devices. Fortunately, several small changes to this design could assist designers with different cost/performance ratios. The simplest and most direct method is to reduce the clock frequency of the MC68030. For instance, if the clock frequency is below approximately 18.1 MHz, the same control logic supports two-clock bus cycles with 45-ns

memory (55 ns if < 15.8 MHz). If 20 MHz is still the frequency of choice, the designer may choose to run three-clock bus cycles. This can be accomplished with the addition of a flip-flop to delay the $\overline{\text{TERM}}$ signal by one clock. The resulting memory access time is over 85 ns with a 20-MHz processor running with three-clock bus cycles.

12.5.2 A 2-1-1-1 Burst Mode Memory Bank Using SRAMs

The MC68030 normally attains its lowest bus utilization when the external memory system can support a 2-1-1-1 burst protocol. However, exceptions to this can occur. For instance, when a large amount of memory accesses are not governed by the locality of reference principles, burst accesses may not decrease bus utilization. This section describes a complete 2-1-1-1 memory bank with 256K bytes that can operate with a 20-MHz MC68030. Nonburst reads and all write cycles execute in two clocks.

Figure 12-14 shows the complete memory bank and its connection to the MC68030. The required parts include:

- (32) 64K $\times$ 1 SRAMs 25 ns access time (Motorola's MCM6287-25 or equivalent)
- (2) 74ALS244 buffers
- (4) 74AS373 latches
- (2) 74F32 OR gates
- (4) 74F191 counters
- (1) PAL16L8D (or equivalent)
- (1) 74F04 inverter

The system must also provide any $\overline{\text{STERM}}$ or $\overline{\text{CBACK}}$ consolidation circuitry as required (e.g., due to the presence of multiple synchronous memory banks or ports). In Figure 12-14, this consolidation circuitry is shown as an AND gate.

The memory bank can be divided into four sections:

1. The byte select and address decode section (provided by the PAL).
2. The burst address generator (provided by the counters).
3. The actual memory section (SRAMs).
4. The buffer section (address and data).

The first section is completely contained within the PAL16L8D. The PAL equations are the same as those provided in Figure 12-8 for the two-clock read, three-clock write memory bank, although slightly modified to support the

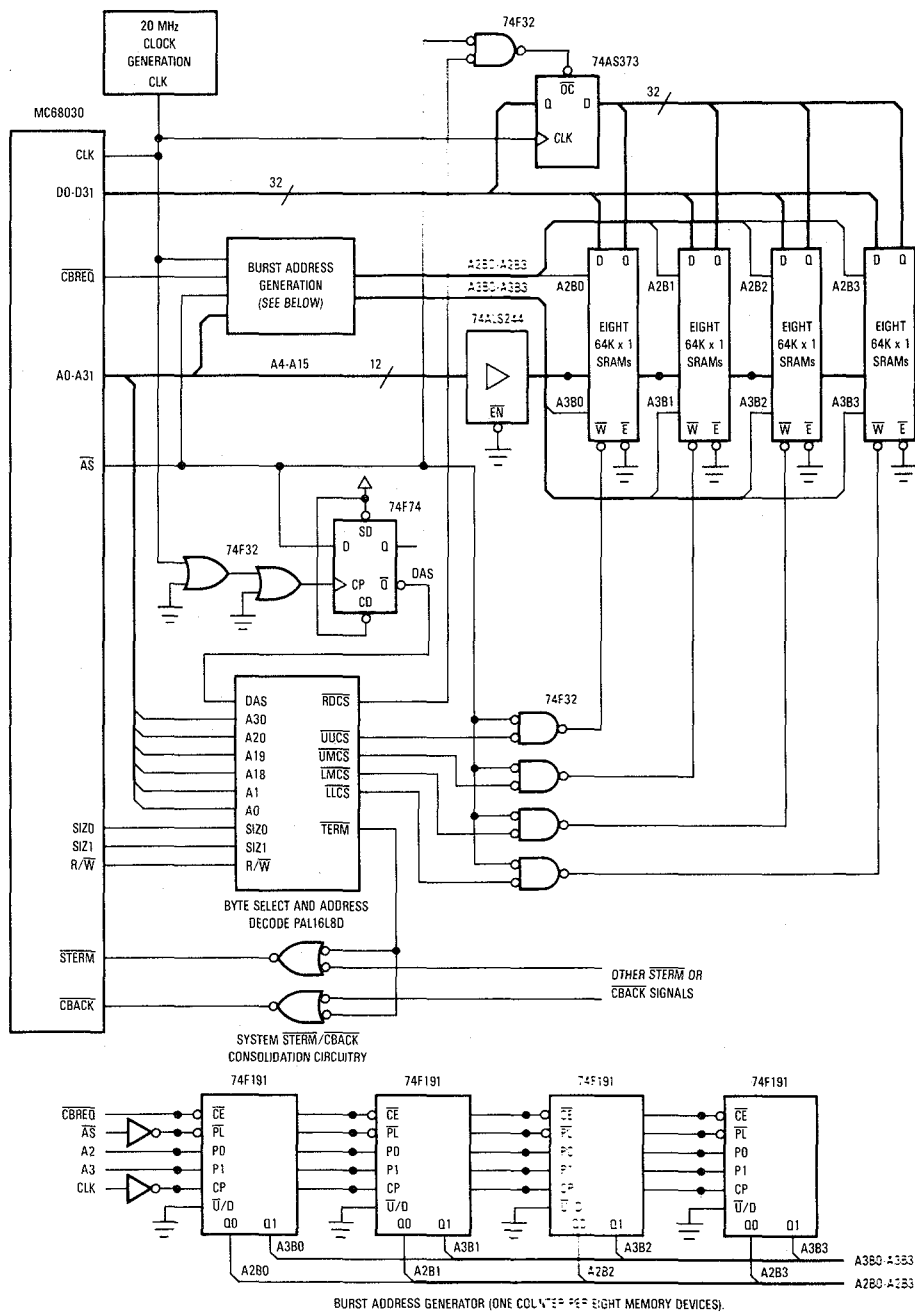


Figure 12-14. Example 2-1-1-1 Burst Mode Memory Bank at 20 MHz, 256K Bytes

larger block of memory (use A18–A20 instead of A16–A18). The PAL generates six memory-mapped signals: four byte select signals for write operations, a buffer control signal, and the cycle termination signal. The byte select signals are only asserted during write operations when the processor is addressing the 256K bytes contained in the memory bank, and then only when the appropriate byte or bytes is being written to as indicated by the SIZ_0 , SIZ_1 , A_0 , and A_1 signals. The four signals, $\overline{UUCS}$, $\overline{UMCS}$, $\overline{LMCS}$, and $\overline{LLCS}$, control data bits D24–D31, D16–D23, D8–15, and D0–D7 respectively. $\overline{AS}$ is used to qualify the byte select signals to avoid spurious writes to memory before the address is valid. During read operations, the read chip select ($\overline{RDCS}$) signal, qualified with $\overline{AS}$, controls the data latches only (since the memory is already enabled with its $\overline{E}$ input grounded). The last signal generated by the PAL is the $\overline{TERM}$ signal. As the equation shows, $\overline{TERM}$ consists of two events: one for read cycles and the other for write cycles. For read cycles, $\overline{TERM}$ is an address decode signal that is asserted whenever the address corresponds to the encoded memory-mapped bank of SRAM. Write operations use the $\overline{DAS}$ signal to qualify the address decode, which lengthens write cycles to three clock periods. If a two-clock write cycle is required, this design can be modified to incorporate the address and data latches used in Figure 12-12. $\overline{TERM}$ is connected to the system's $\overline{STERM}$ and $\overline{CBACK}$ consolidation circuitry such that both are asserted when $\overline{TERM}$ is asserted. The consolidation circuitry should have a maximum propagation delay of 15 ns or less. If the system has no other synchronous memory or ports, $\overline{TERM}$ can be connected directly to $\overline{STERM}$, and $\overline{CBACK}$ may be grounded.

The second section is the burst address generator which contains the four counters and the inverter. The counters serve to both buffer the MC68030's address lines (A_2 and A_3) and to provide the next long-word address during a burst operation. The 74F191s are asynchronously preset at the beginning of every bus cycle when $\overline{AS}$ is negated. When $\overline{AS}$ asserts, the counting is dependent on the $\overline{CBREQ}$ signal and the CLK signal. During writes, $\overline{CBREQ}$ is always negated, and the counters serve only as address buffers. During reads, if $\overline{CBREQ}$ asserts, the current value of counter bits Q1:Q0 are incremented on every falling clock edge of the MC68030's clock after $\overline{AS}$ asserts. Four counters are used to provide enough drive capability to avoid an additional buffer propagation delay. Each counter drives eight memory devices.

The third section contains the memory devices. The most important feature of the memory devices used in this design is the separate data-in and data-out pins, which allow the SRAMs to be constantly enabled before address decode is complete without causing data bus contention. If the designer

wishes to include some type of enable circuitry to take advantage of low bus utilization, the timing in this design will be preserved if the memory's $\bar{E}$ signal is asserted within 13 ns after the falling edge of state S0.

The fourth and last section of the memory bank is the address and data buffers. The address buffers are shown as 74ALS244s, but 74AS244s and 74F244s are also acceptable. Two inputs to the address buffers remain unused allowing the possibility for expansion up to 1 Mbyte without any additional devices when SRAMs of suitable density become available. The $\bar{RD}\overline{CS}$ signal, qualified with $\overline{AS}$, controls the data buffers during read operations. The address buffers are always enabled.

Some modifications to this design can improve performance. Specifically, circuitry to control CBACK and thus prevent or discontinue a burst cycle is a simple addition. The circuitry should have two functions: to prevent wrap-around and to prevent bursting when a data operand crosses a long-word boundary.

Not all systems require the performance of 20-MHz 2-1-1-1 burst cycles, nor will all systems be able to afford the fast devices of this design. If the clock frequency is below approximately 17.5 MHz, the same support logic supports 2-1-1-1 burst cycles with 35-ns memory. If 20 MHz is still the frequency of choice, the designer may choose to run 3-1-1-1 burst cycles.

12.5.3 A 3-1-1-1 Burst Mode Memory Bank Using SRAMs

Figure 12-15 shows the complete 3-1-1-1 memory bank with 256K bytes that can operate with a 20-MHz MC68030. The required parts include:

- (32) 64K $\times$ 1 SRAMs 35-ns access time (Motorola's MCM6287-35 or equivalent)
- (4) 74ALS244 buffers
- (4) 74F374 latches
- (2) 74F32 OR gates
- (4) 74F191 counters
- (1) PAL16L8D (or equivalent)
- (2) inverters
- (1) flip-flop

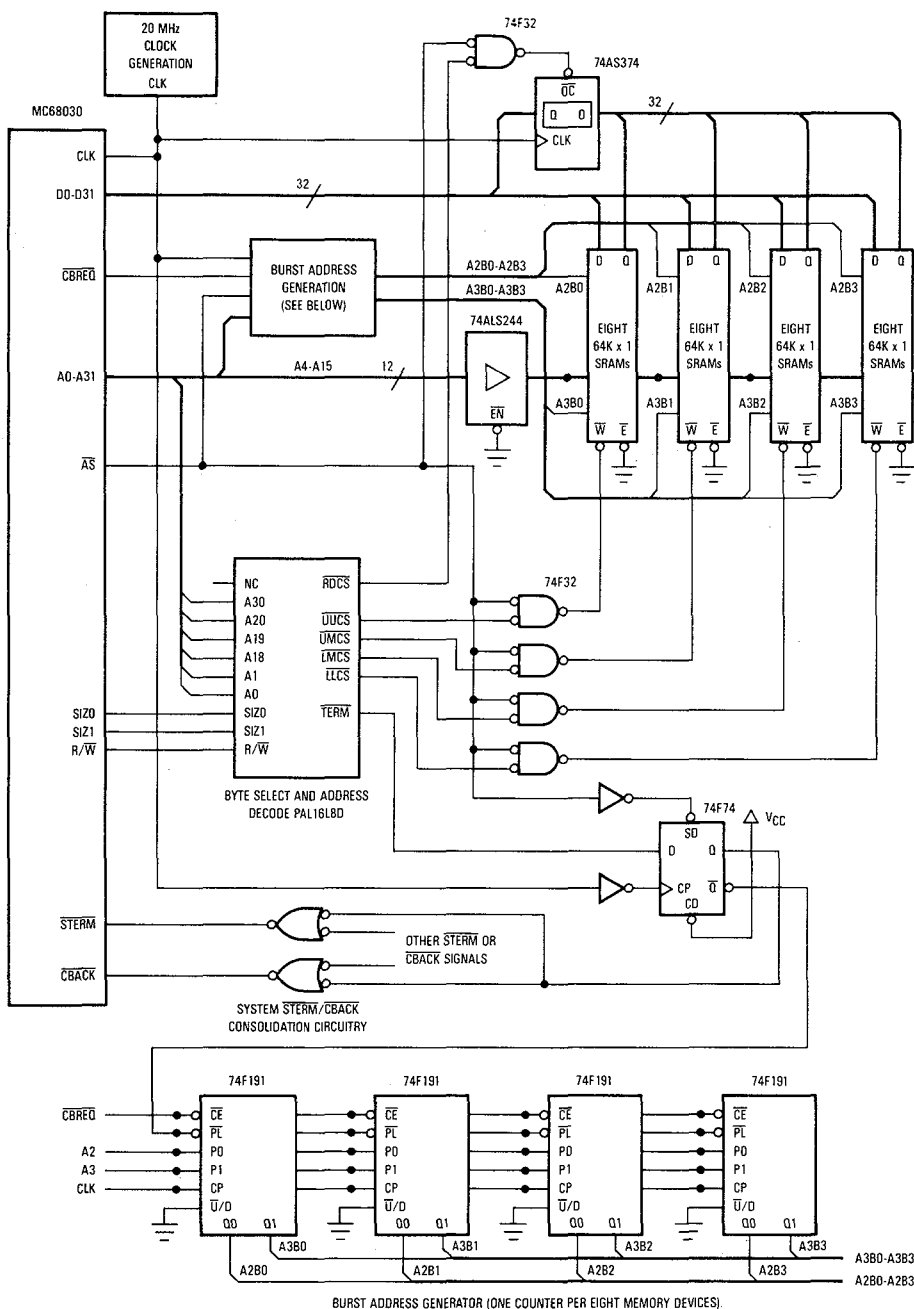


Figure 12-15. Example 3-1-1-1 Pipelined Burst Mode Memory Bank at 20 MHz, 256K Bytes

The structure of this memory bank is very similar to the 2-1-1-1 memory bank described in **12.5.2 A 2-1-1-1 Burst Mode Memory Bank Using SRAMs**. In fact, the PAL and address buffers are exactly the same. The PAL equations are provided in Figure 12-10. The most important differences occur in the data latches, which are now flip-flops. Also, the D-type flip-flop has been moved from the input side of the PAL to the $\overline{\text{TERM}}$ output.

The data flip-flops allow the long words out of the memory to be pipelined such that setup and hold times are easier to satisfy. The memory devices are generating the next long word of data even before the MC68030 has latched the "current" long word. This alteration eases access timing requirements such that 35-ns memory can be used with a clock frequency of 20 MHz. If the clock frequency is less than 17 MHz, 45-ns memory can be used. Another benefit of the slower cycle is a relaxed timing requirement for the enable inputs of the SRAMs. Although Figure 12-15 has all the SRAM chip enables grounded, the timing in this design will be preserved if the memory's $\overline{\text{E}}$ signal is asserted within 10 ns after the rising edge of state S2. Figure 12-16 shows four possible enable circuits.

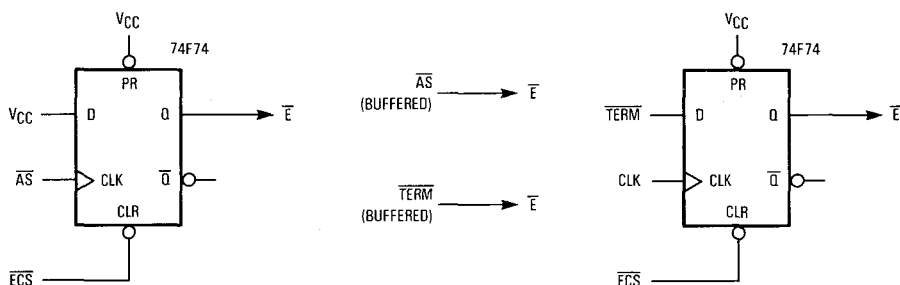


Figure 12-16. Additional Memory Enable Circuits

The flip-flop connected to the $\overline{\text{TERM}}$ signal serves two purposes: first, the $\overline{\text{TERM}}$ signal is delayed at the beginning of the cycle to insert the wait state for the first long word, and second, the burst address generator is also prevented from incrementing the long word base address until the first long word has been latched by the 74F374s.

The performance enhancing modifications described for the 2-1-1-1 design also apply to this design. Specifically, circuitry can be added to control $\overline{\text{CBACK}}$ and thus prevent or discontinue a burst cycle. The circuitry should have two functions: first, to prevent wraparound and second, to prevent bursting when

a data operand crosses a long-word boundary. Another enhancement might be to alter the $\overline{\text{TERM}}$ control circuitry with the addition of a write latch mechanism to run two-clock writes.

The critical path for the 3-1-1-1 memory bank is not the first long-word access as in the 2-1-1-1 memory bank, but rather the subsequent long words during burst cycles. No alternative architecture can correct the critical path for the 2-1-1-1 burst cycle. However, for 3-1-1-1 burst cycles, the designer might consider memory banks which are 64 or 128 bits wide. In this manner, the access time for the subsequent long words can be hidden underneath the access of the previous long word(s).

12.6 EXTERNAL CACHES

To provide lower average access times to memory, some systems implement caches local to the main processor that store recently used instructions and/or data. For the MC68030, several architectural options are available to the cache designer. The primary decisions are whether to configure the cache as an asynchronous or synchronous device and whether the cache accesses are terminated early (before the cache lookup is complete) or only after validation.

The MC68030 late $\overline{\text{BERR}}/\overline{\text{HALT}}$ facility allows an external device to signal completion of a bus cycle by asserting $\overline{\text{DSACKx}}$ or $\overline{\text{STERM}}$ and later (approximately one clock period or one-half clock, respectively) aborting or retrying that cycle if an error condition is detected. Since one critical access path in many memory structures is the assertion of $\overline{\text{DSACKx}}/\overline{\text{STERM}}$ to avoid additional wait states, the late abort capability allows the memory controller to terminate a bus cycle before data is valid on the processor data bus. If the data validation fails, the memory controller can then abort ($\overline{\text{BERR}}$) or retry ($\overline{\text{BERR}}/\overline{\text{HALT}}$) the cycle. This technique is useful in memory error detection schemes where the cycle can be terminated as soon as data becomes available and the error checking can occur during the period between the signaling of termination of the cycle and the latching of data by the processor with a late retry or abort signaled upon error indication. Likewise, this technique can be used in cache implementations in which the cache tag validation cannot be completed before termination of the cycle must be signaled but the validation is completed before late abort or retry must be indicated.

The major consideration in choosing whether or not to utilize late retry for an external cache miss is the overhead involved in retrying a bus cycle after a miss in the cache. The minimum penalty is the four clock periods required

to retry the cycle (two clocks during which the miss is detected and two clocks idle bus time), assuming that the bus control strobes ($\overline{\text{BERR}}$ and $\overline{\text{HALT}}$) are negated soon enough after the completion of the aborted cycle that the next cycle can begin immediately. In evaluating this overhead, the projected cache miss rate determines the percentage of cycles that must be retried. Additionally, the degree of parallelism in the system should be considered. If, after a cache miss, it is possible to continue the bus cycle to main memory while the processor is retrying the cycle, it is possible to avoid some, or all, of the performance penalty associated with late retry (although the control circuitry required may be more complex).

For a two-clock bus or burst capability, use of the synchronous bus is mandated, but for a three or more clock, nonburst cache, the choice of synchronous versus asynchronous operation must be made. If the bus cycle is terminated only after validation, use of the synchronous bus is recommended since the address-valid-to- $\overline{\text{STERM}}$ -asserted timing requirement is longer than the address-valid-to- $\overline{\text{DSACK}}$ -asserted timing for bus cycles of the same length. If the cache implements late retry, the choice of which bus control mode to use is less important and depends on system-specific features and control structures. Some external caches might use both synchronous and asynchronous transfers: synchronous for hits and asynchronous for misses or vice versa. The following discussion assumes that the external cache uses the synchronous two-clock protocol, but most statements also apply to the asynchronous protocol.

If the MC68030 MMU is disabled, all bus cycles use logical addresses. If the MMU is enabled, the external address bus uses physical addresses (including directly mapped logical-to-physical addresses from the transparent translation (TTx) registers). These two modes of operation, logical and physical, affect the maintenance of external caches. For example, when the external cache uses physical addresses, the cache need not be flushed on each context switch. Since each task in a system may have its own unique mapping of the logical address space, a logical cache must be flushed of all entries any time the logical-to-physical mapping of the system changes (as occurs during a context switch). Since there is only a single physical address space, this problem does not occur with a physical cache because all references to a particular operand must utilize the same physical address.

The intended cache size should be evaluated when considering the utility of allowing multiple tasks to maintain cache entries. If the cache is relatively small and the time between context switches is large, each task will tend to fill the cache and remove all entries created during the execution of previous

tasks. Conversely, if the cache size is relatively large and the period between context switches is relatively small, the cache may provide an efficient sharing of entries.

12.6.1 Cache Implementation

An example organization of an external cache is shown in Figure 12-15. With this organization, the cache timing controller does not terminate a bus cycle until the cache has had sufficient time to validate the access as a “hit” or a “miss”. When a hit decision is made, the cache controller asserts the $\overline{\text{STERM}}$ signal and also blocks propagation of $\overline{\text{AS}}$ (A) to the external system. If the cache decision cannot be completed before $\overline{\text{AS}}$ would normally be asserted by the MC68030, some provision must be made to delay the propagation of $\overline{\text{AS}}$ until the decision is valid. Otherwise, spurious assertions of the $\overline{\text{AS}}$ signal are likely to occur.

The cache control circuit (B) contains all logic required to clear or create cache entries. Also contained in (B) is the decision logic required to determine whether a hit or miss has occurred and the timing logic that is required to prevent propagation of the “hit” signal until the lookup and compare circuitry has had sufficient time to generate a valid decision. The critical path in the design of this cache is from the output of valid address by the MC68030 to the assertion of $\overline{\text{STERM}}$ by the cache controller (see Equation 12-3 of Table 12-2). After a cache hit decision has been made, the hit signal directly drives the $\overline{\text{STERM}}$ signal. Qualifying $\overline{\text{STERM}}$ with $\overline{\text{AS}}$ is not necessary assuming the appropriate setup and hold times are respected when $\overline{\text{AS}}$ is asserted. Operating at 20 MHz with no wait states, 21 ns are available from the presentation of valid address by the MC68030 to the assertion of $\overline{\text{STERM}}$ by the cache controller while 46 ns are available from valid address to data valid at the processor.

12

If the access times cannot be met due to the particular cache architecture, size, cost, or other consideration, the system designer may choose to utilize an early termination approach, as discussed above, that increases the decision time available to the cache controller by meeting the critical path from address valid to $\overline{\text{BERR}}/\overline{\text{HALT}}$ asserted (see Equation 12-5 of Table 12-2). The only required changes to the cache structure shown in Figure 12-17 is the generation of $\overline{\text{STERM}}$. Figure 12-18 shows an example circuit that could be positioned between the MC68030 and the external cache to provide the early termination or late retry function.

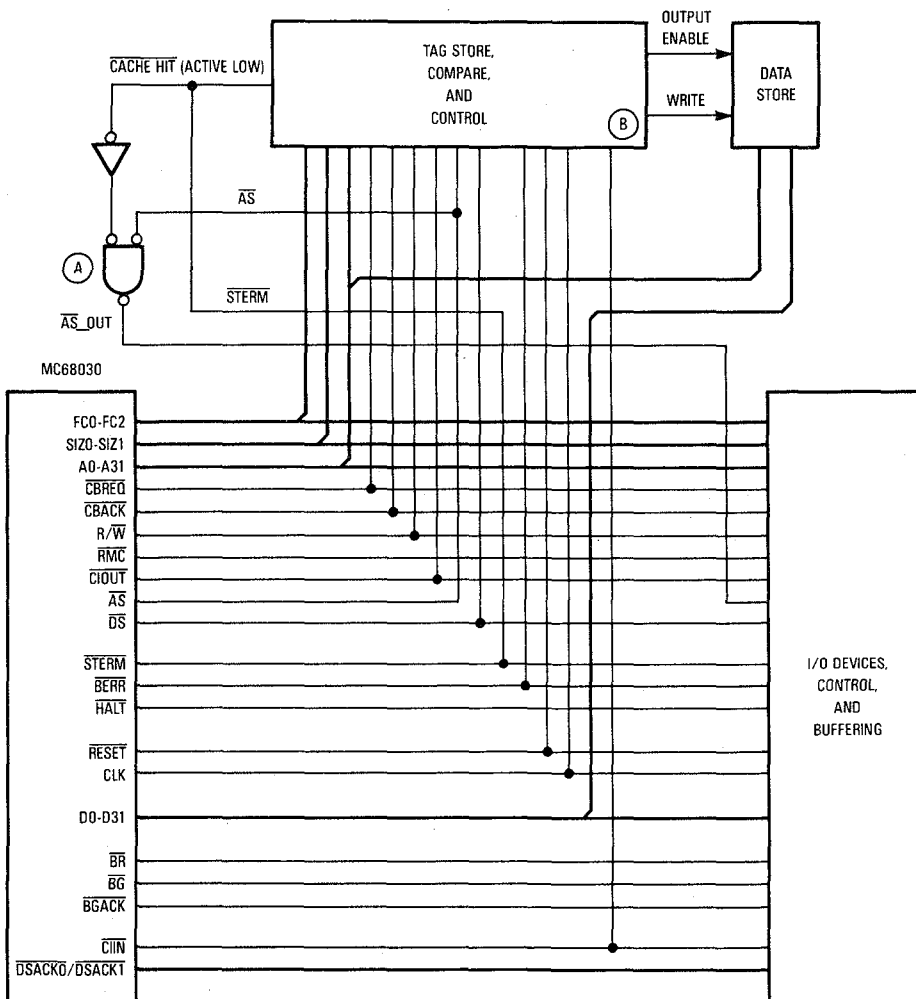
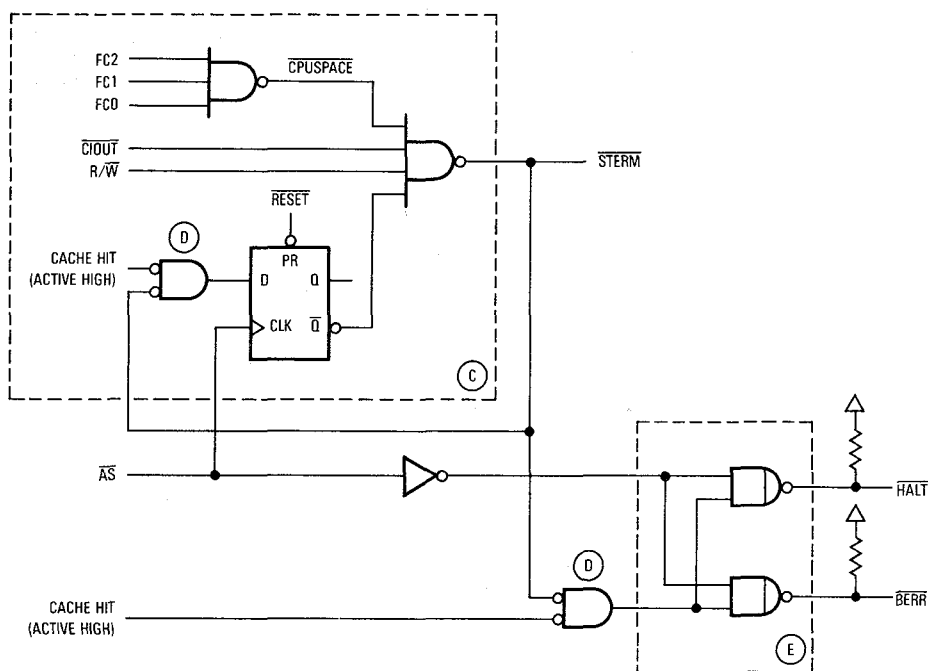


Figure 12-17. Example MC68030 Hardware Configuration with External Physical Cache

Normally, as soon as $\overline{AS}$ is asserted, circuit (C) immediately asserts the $\overline{STERM}$ signal to terminate the bus cycle, assuming that the cache will produce a valid hit later in the cycle. Circuit (C) also prevents the early termination from occurring from those cycles that access operands that are noncacheable or had missed in the cache on the previous cycle (and have not already been retried). In this example, (C) prevents early termination of all CPU space accesses, all write cycles (assuming a writethrough cache is implemented),



To minimize the potential for delays in retrying a bus cycle, the negation path of the bus error and halt signals should be carefully controlled. Light capacitive loading of these signals lines as well as the use of a properly sized pullup resistor for any open collector drivers, or some equivalent method, is recommended.

The available cache tag lookup, compare, and logic delay (D) and (E) time for this implementation is given by Equation 12-5 of Table 12-2 (40 ns at 20.0 MHz with no wait states).

A further design consideration is the response of the main memory controller to accesses that miss in the cache and are retried. During a retry operation and in the absence of arbitration for the logical bus, the MC68030 continuously drives the address bus with the address that caused the retry to be signaled. This presents the designer with the opportunity to utilize this information to continue (or initiate) the access in the main memory (by latching the state of the $\overline{AS}$ signal during the initial bus cycle and holding it asserted for the duration of the retry), thus decreasing the overhead associated with retrying the cycle.

12.6.2 Instruction-Only External Cache Implementations

In some cases, particularly in multiprocessing systems where cache coherence is a concern, it is desirable to store only instruction operands since they are not considered to be alterable and, hence, cannot generate stale data. In general, this is feasible with the MC68000 architecture as long as PC relative addressing modes are not used. This restriction allows program and data accesses to be distinguished externally by decoding the function code signals.

12.7 DEBUGGING AIDS

The MC68030 supports the monitoring of internal microsequencer activity with the $\overline{STATUS}$ and $\overline{REFILL}$ signals. The use of these signals is described in the following paragraph. A useful device to aid programming debugging is described in **12.7.2 Real-Time Instruction Trace**.

12.7.1 STATUS and REFILL

The MC68030 provides the STATUS and REFILL signals to identify internal microsequencer activity associated with the processing of data in the pipeline. Since bus cycles are independently controlled and scheduled by the bus controller, information concerning the processing state of the microsequencer is not available by monitoring bus signals by themselves. The internal activity identified by the STATUS and REFILL signals include instruction boundaries, some exception conditions, when the microsequencer has halted, and instruction pipeline refills. STATUS and REFILL track only the internal microsequencer activity and are not directly related to bus activity.

As shown in Table 12-4, the number of consecutive clocks during which STATUS is asserted indicates an instruction boundary, an exception to be processed, or that the processor has halted. Note that the processor halted condition is an internal error state in which the microsequencer has shut itself down due to a double bus fault and is not related to the external assertion of the HALT input signal. The HALT signal only affects bus operation, not the microsequencer.

Table 12-4. Microsequencer STATUS Indications

Asserted for	Indicates
1 Clock	Sequencer at instruction boundary will begin execution of next instruction
2 Clocks	Sequencer at instruction boundary but will not begin next instruction immediately due to: • pending trace exception - OR • pending interrupt exception
3 Clocks	MMU address translation cache miss — processor to begin table search OR Exception processing to begin for: • reset OR • bus error OR • address error OR • spurious interrupt OR • autovector interrupt OR • F-line instruction (no coprocessor responded)
Continuously	Processor halted due to double bus fault

The REFILL signal identifies when the microsequencer requests an instruction pipeline refill. Refill requests are a result of having to break sequential instruction execution to handle nonsequential events. Both exceptions and instructions can cause the assertion of REFILL. Instructions that cause refills include branches, jumps, instruction traps, returns, coprocessor general in-

instructions that modify the program counter flow, and status register manipulations. Logical and arithmetic operations affecting the condition codes of the status register do not result in a refill request. However, operations like the `MOVE <ea>,SR` instruction, which updates the status register, cause a refill request since this can change the program space as defined by the function codes. When the program space changes, the processor must fetch data from the new space to replace data already prefetched from the old program space. Similarly, operations which affect the address translation mechanism of the memory management unit (MMU) cause a refill request. An instruction like the `PMOVE <ea>,TC`, which changes the translation control register, requires the processor to fetch data from the new address translation base. The Test Condition, Decrement, and Branch (DBcc) instruction causes two refill requests when the condition being tested is false. To optimize branching performance, the DBcc instruction requests a refill before the condition is tested. If the condition is false, another refill is requested to continue with the next sequential instruction.

Figure 12-19 illustrates the relation between the CLK signal and normal instruction boundaries as identified by the `STATUS` signal. `STATUS` asserting for one clock cycle identifies normal instruction boundaries. Note that the assertion of `REFILL` does not necessarily correspond to the assertion of `STATUS`. Both `STATUS` and `REFILL` assert and negate from the falling edge of the CLK signal.

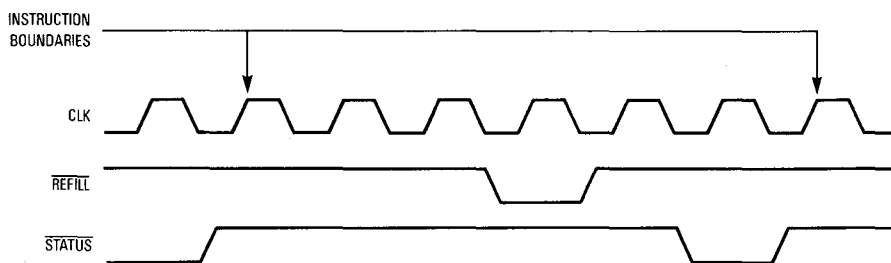


Figure 12-19. Normal Instruction Boundaries

Figure 12-20 shows a normal instruction boundary followed by a trace or interrupt exception boundary. `STATUS` asserting for two clock cycles identifies a trace or interrupt exception. Instruction boundary information is still present since both trace and interrupt exceptions are processed only at instruction boundaries. Before the exception handler instructions are prefetched, the `REFILL` signal asserts (not shown) to identify a change in program flow.

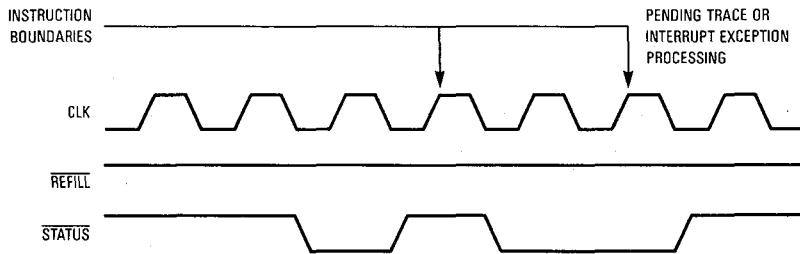


Figure 12-20. Trace or Interrupt Exception

Figure 12-21 illustrates the assertion of the $\overline{\text{STATUS}}$ signal for other exception conditions, which include MMU address translation cache miss, reset, bus error, address error, spurious interrupt, autovector interrupt, and F-line instruction when no coprocessor responds. Exception processing causes $\overline{\text{STATUS}}$ to assert for three clock cycles to indicate that normal instruction processing has stopped. Instruction boundaries cannot be determined in this case since these exceptions are processed immediately, not just at instruction boundaries.

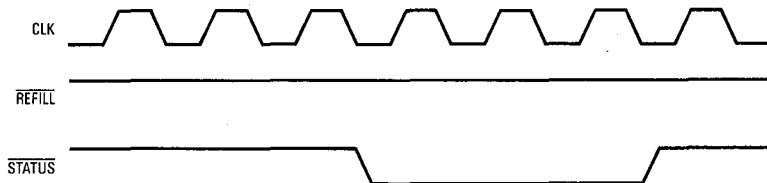


Figure 12-21. Other Exceptions

Figure 12-22 shows the assertion of $\overline{\text{STATUS}}$, indicating that the processor has halted due to a double bus fault. Once a bus error has occurred, any additional bus error exception occurring before the execution of the first instruction of the bus error handler routine constitutes a double bus fault. The processor also halts if it receives a bus error or address error during the vector table read operations or the prefetch for the first instruction after an external reset. $\overline{\text{STATUS}}$ remains asserted until the processor is reset.

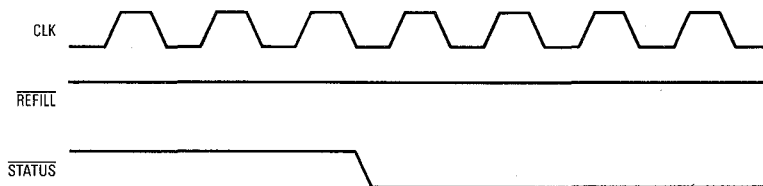


Figure 12-22. Processor Halted

12.7.2 Real-Time Instruction Trace

Microprocessor-based systems used for real-time applications typically lack development aids for program debug. The real-time environment does not allow program instruction execution to arbitrarily stop to handle debugging events. These systems include control applications where mechanical events cannot halt, such as robotics, automotive, and industrial control and emulator systems which may need to keep the target system executing in real time.

To solve the problems inherent with real-time systems, the MC68030 incorporates extra hardware-based features to enhance program debug. Real-time systems cannot take advantage of the trace exception mechanism built into all M68000 Family processors since this takes processing time away from real-time events. Additional output pins have been incorporated into the MC68030 to gain real-time visibility into the processor. Tracing capability can be added by decoding MC68030 control signals to detect which cycles are important for tracking. Post analysis of collected data allows for program debug.

Several problems exist with an external trace mechanism. These problems include determining which cycles are important for tracking program flow, detecting if instructions obtained in prefetch operations are discarded by the execution unit, and the inability of external trace circuitry to capture accesses to on-chip cache memories.

External trace hardware used for program debug must be synchronized to the MC68030 bus activity. Since all clock cycles are not traced in a program debug environment, the trace hardware requires a sampling signal. For external read and write operations, trace sampling occurs when the data bus contains valid data. Two modes of external bus operation are possible: the synchronous mode in which the system returns the `STERM` signal and the asynchronous mode in which the system responds with the `DSACK1` and or the `DSACK0` signals. Both modes of bus operation need to generate a sam-

pling signal when valid data is present on the bus. This allows for tracing data flow in and out of the processor, which is the basis for tracking program execution.

The pipelined architecture of the MC68030 prefetches instructions and operands to keep the three stages of the instruction pipe full. The pipeline allows concurrent operations to occur for up to three words of a single instruction or for up to three consecutive instructions. While sequential instruction execution is the norm, it is possible that prefetched data is not used by the execution unit due to a nonsequential event. The STATUS signal allows trace hardware to mark the progress of the execution unit as it processes program memory operands and allows marking of some exceptions. Nonsequential events, where the entire pipeline needs to reload before continuing execution, are marked by the REFILL signal.

External hardware typically has no visibility into on-chip cache memory operations. However, the MC68030 provides a local address reference to increase visibility. Write operations are totally visible since the MC68030 implements a writethrough policy allowing external hardware to capture data. For read operations from on-chip cache memories, the least significant byte of the address bus provides a local address reference.

The MC68030 begins an external cycle by driving the address bus and asserting the external cycle start (ECS) signal. Address strobe (AS) asserts later in the cycle to validate the address. If a hit occurs in the cache or the cache holding register, then the external cycle is aborted and AS is not asserted. In addition, the low-order address bits (A0–A7) are not involved in the address translation process performed by the on-chip MMU, creating a local address reference which can be used by trace functions. All read cycles from the on-chip cache memories cannot be captured externally since the cache access does not depend on the availability of the external bus.

Figure 12-23 shows a trace interface circuit which can be used with a logic analyzer for program debug. The nine input signals (DSACK1, DSACK0, CLK, AS, RESET, STATUS, REFILL, STERM, and ECS) are connected to the MC68030 processor in the system under development. Six output signals are generated to aid in capturing and analyzing data. In addition to connecting the logic analyzer to the address bus, the data bus, and the bus control signals, the trace interface signals (SAMPLE, PHALT, FILL, EP, IE, and ECSC) should also be connected. The external clock probe of the logic analyzer connects to the system CLK signal for synchronization. Setting up the logic analyzer for data capture requires that samples be taken on the falling edge of the CLK signal when the SAMPLE signal is high. Table 12-5 lists the parts required to implement this circuit.

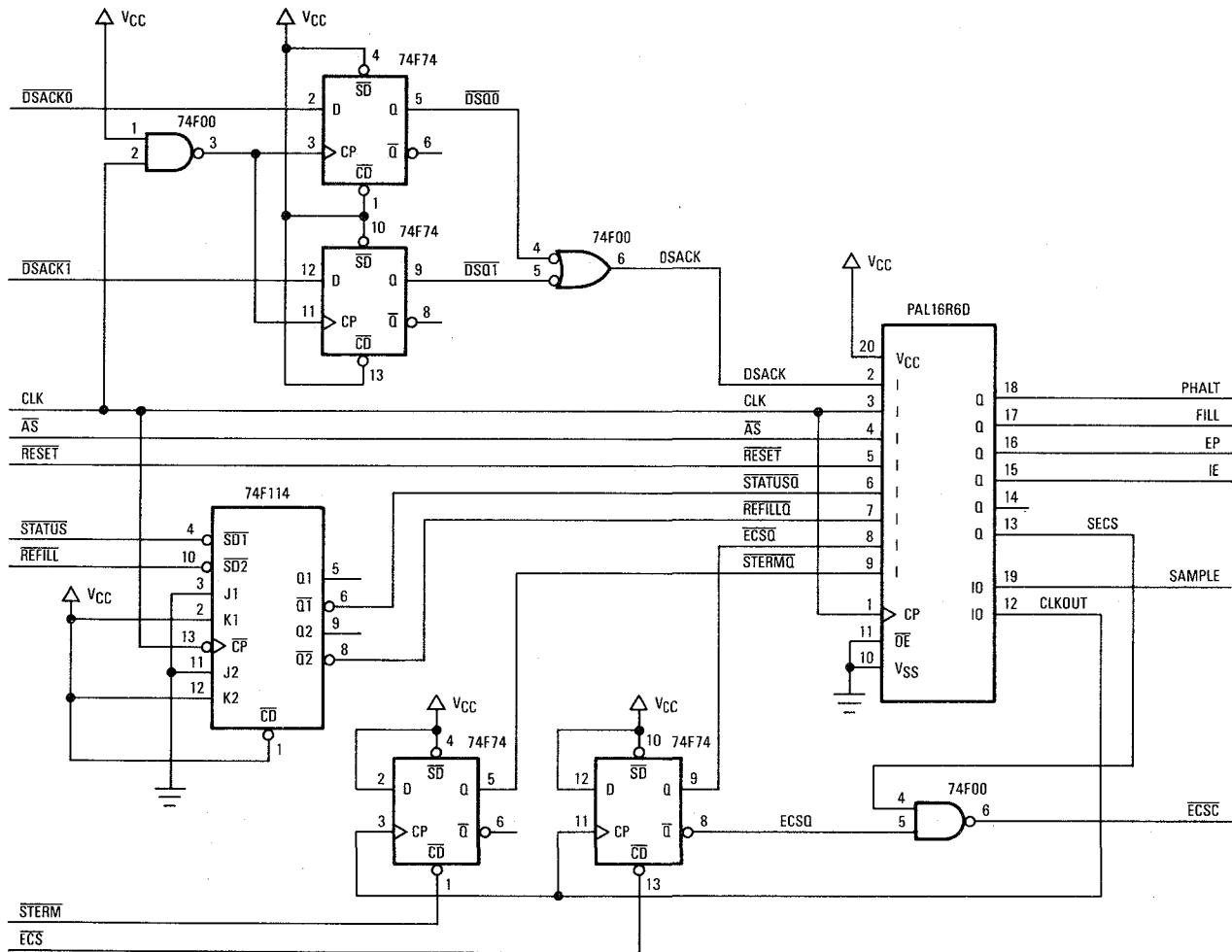


Figure 12-23. Trace Interface Circuit

Table 12-5. List of Parts

Quantity	Part	Part Description
1	74F00	Quad 2 Input NAND Gate
1	74F114	Dual JK Negative Edge-Triggered Flip-Flop
2	74F74	Dual D-Type Positive Edge-Triggered Flip-Flop
1	PAL16R6D	Programmable Logic Array, Ultra High Speed

The sample signal (SAMPLE) is an active-high signal which qualifies the next falling edge of the CLK signal as the sampling point. Five types of conditions cause SAMPLE to assert:

1. An external bus cycle
2. An internal cache hit, including a hit in the cache holding register
3. An instruction boundary
4. Exception processing as marked by the EP signal discussed below
5. The processor halting

The remaining five output signals are used to qualify the information collected.

The processor halt (PHALT) signal indicates that the MC68030 has received a double bus fault and needs a reset operation to continue processing. PHALT asserts after the assertion of STATUS for greater than three clock cycles and generates a SAMPLE signal.

The FILL signal indicates a break in sequential instruction execution. FILL is a latched version of the REFILL signal and remains asserted until a sample is collected as indicated by the assertion of SAMPLE. The assertion of FILL does not generate a SAMPLE signal.

The exception pending (EP) signal indicates that the MC68030 is beginning exception processing for either a reset, bus error, address error, spurious interrupt, autovector interrupt, F-line instruction, MMU address translation cache miss, trace exception, or interrupt exception. The EP signal asserts after STATUS negates from a two- or three-clock cycle assertion. The assertion of EP does generate a SAMPLE signal.

The instruction executed (IE) signal indicates the execution unit has just finished processing an instruction. The IE signal asserts after STATUS negates from a one-clock cycle assertion. The assertion of IE also generates a SAMPLE signal.

The external cycle start condition ($\overline{\text{ECSC}}$) signal is used in conjunction with the $\overline{\text{AS}}$ signal to determine if the address bus and data bus are valid in the current trace sample. Table 12-6 lists the possible combinations of $\overline{\text{AS}}$ and $\overline{\text{ECSC}}$ and shows what parts of the traced address and data bus are valid. The assertion of $\overline{\text{ECSC}}$ does not generate a SAMPLE signal.

Table 12-6. $\overline{\text{AS}}$ and $\overline{\text{ECSC}}$ Indications

$\overline{\text{AS}}$	$\overline{\text{ECSC}}$	Indicates
0	0	Both Address and Data Bus Are Valid
0	1	Both Address and Data Bus Are Valid
1	0	Address Bits (A0–A7) are Valid Address Bits (A8–A31) Are Invalid Data Bus Is Invalid
1	1	Both Address and Data Bus Are Invalid

Figure 12-24 shows the pin definitions for the PAL16R6 package used in the trace circuit. These definitions are used by the PAL equations listed in Figure 12-25.

12.8 POWER AND GROUND CONSIDERATIONS

The MC68030 is fabricated in Motorola's advanced HCMOS process, contains approximately 275,000 total transistor sites, and is capable of operating at clock frequencies of up to 33.33 MHz. While the use of CMOS for a device containing such a large number of transistors allows significantly reduced power consumption in comparison to an equivalent NMOS circuit, the high clock speed makes the characteristics of power supplied to the device very important. The power supply must be able to supply large amounts of instantaneous current when the MC68030 performs certain operations, and it must remain within the rated specification at all times. To meet these requirements, more detailed attention must be given to the power supply connection to the MC68030 than is required for NMOS devices that operate at slower clock rates.

```

/*****/
/* This device generates a sampling signal for tracing processor activity on          *
/* an instruction level basis for the MC68030. In the pin definitions and            *
/* equations listed below the following symbols are used:                          *
/*                                     Symbol  Definition                          *
/*                                     !       Logical NOT                          *
/*                                     #       Logical OR                           *
/*                                     &       Logical AND                          *
/* In addition, the '.d' extension on signal names refers to the 'D' input of      *
/* the internal PAL flip flop.                                                     *
/*****/
/* Allowable Target Device Types : PAL16R6D High Speed PAL                       *
/*****/
/**   Inputs   **/
PIN 1   = clk           ; /* same as pin 3 CLK                      */
PIN 2   = DSACK         ; /* Data Strobe Acknowledge          */
PIN 3   = CLK           ; /* MPU Clock Signal                 */
PIN 4   = !AS           ; /* Address Strobe                   */
PIN 5   = !RESET        ; /* System Reset Signal              */
PIN 6   = !STATUSQ      ; /* Latched STATUS Signal           */
PIN 7   = !REFILLQ      ; /* Latched REFILL Signal           */
PIN 8   = !ECSQ         ; /* Latched ECS Signal              */
PIN 9   = !STERMQ       ; /* Latched STERM Signal            */

/**   Outputs  **/
PIN 19  = SAMPLE        ; /* Sample Signal                    */
PIN 18  = PHALT         ; /* Processor Halted                 */
PIN 17  = FILL          ; /* REFILL received                  */
PIN 16  = EP            ; /* Exception Pending                */
PIN 15  = IE            ; /* Instruction Executed              */
PIN 14  = sc            ; /* status complete                  */
PIN 13  = secs          ; /* sampled ECS signal               */
PIN 12  = CLKOUT        ; /* Delayed CLK Signal              */

```

Figure 12-24. PAL Pin Definitions

To supply a solid power supply interface, 10 V_{CC} pins and 14 GND pins are provided. This allows two V_{CC} and four GND pins to supply power for the address bus and two V_{CC} and four GND pins to supply the data bus; the remaining V_{CC} and GND pins are used by the internal logic and clock generation circuitry. Table 12-7 lists the V_{CC} and GND pin assignments.

To reduce the amount of noise in the power supplied to the MC68030 and to provide for instantaneous current requirements, common capacitive decoupling techniques should be observed. While there is no recommended layout for this capacitive decoupling, it is essential that the inductance between these devices and the MC68030 be minimized to provide sufficiently fast response time to satisfy momentary current demands and to maintain

```

/** Intermediate Equations */
S0 = !PHALT & !SC & !EP & !IE; /* State = PHALT SC EP IE */
S1 = !PHALT & !SC & !EP & IE; /* 0 = 0 0 0 */
S2 = !PHALT & !SC & EP & IE; /* 1 = 0 0 1 */
S3 = !PHALT & !SC & EP & !IE; /* 2 = 0 0 1 */
S4 = PHALT & SC & EP & IE; /* 3 = 0 0 1 */
S5 = !PHALT & SC & !EP & IE; /* 4 = 1 1 1 */
S6 = !PHALT & SC & EP & IE; /* 5 = 0 1 0 */
S7 = !PHALT & SC & EP & !IE; /* 6 = 0 1 1 */
/* 7 = 0 1 0 */

/** Logic Equations */
!SAMPLE = !SC & !AS & !SECS #
          !SC & !DSACK & !STERMQ & !SECS #
          !SC & AS & !DSACK & !STERMQ & SECS;

!PHALT.d = !STATUSQ # !EP # IE # RESET;

!SC.d = RESET #
        S0 #
        S1 & STATUSQ #
        S2 & STATUSQ #
        S4 & !STATUSQ #
        SC & !PHALT;

!EP.d = RESET #
        S0 #
        S1 & !STATUSQ #
        S4 & !STATUSQ #
        SC & !PHALT;

!IE.d = RESET #
        S0 & !STATUSQ #
        S2 & STATUSQ #
        S3 & !STATUSQ #
        SC & !STATUSQ;

!SECS.d = !ECSQ;

!CLKOUT = !CLK;

!FILL.d = !REFILLQ & SAMPLE #
          !FILL & !REFILLQ #
          RESET;

```

Figure 12-25. Logic Equations

Table 12-7. V_{CC} and GND Pin Assignments

Pin Group	V _{CC}	GND
Address Bus	C6, D10	C5, C7, C9, E11
Data Bus	L6, K10	J11, L9, L7, L5
$\overline{ECS}$, SIZ_x , $\overline{DS}$, $\overline{AS}$, $\overline{DBEN}$, $\overline{CBREQ}$, $R\overline{W}$	K4	J3
FC0–FC2, $\overline{RMC}$, $\overline{OCS}$, $\overline{CIOUT}$, $\overline{BG}$	D4	E3
Internal Logic, $\overline{RESET}$, STATUS, REFILL, Misc.	H3, F2, F11, H11	L8, G3, F3, G11

a constant supply voltage. It is suggested that a combination of low, middle, and high frequency, high-quality capacitors be placed as close to the MC68030 as possible (e.g., a set of 10 μ F, 0.1 μ F, and 330 pF capacitors in parallel provides filtering for most frequencies prevalent in a digital system). Similar decoupling techniques should also be observed for other VLSI devices in the system.

In addition to the capacitive decoupling of the power supply, care must be taken to ensure a low-impedance connection between all MC68030 V_{CC} and GND pins and the system power supply planes. Failure to provide connections of sufficient quality between the MC68030 power supply pins and the system supplies will result in increased assertion delays for external signals, decreased voltage noise margins, and potential errors in internal logic.